

DeepSeek-V4: 迈向高效百万级上下文智能

DeepSeek-AI
research@deepseek.com

摘要

我们推出了DeepSeek-V4系列的预览版，其中包括两个强大的混合专家（Mixture-of-Experts, MoE）语言模型——DeepSeek-V4-Pro，参数为1.6T（激活参数为49B），以及DeepSeek-V4-Flash，参数为284B（激活参数为13B）——两者均支持一百万个标记的上下文长度。DeepSeek-V4系列在架构和优化方面进行了多项关键升级：（1）混合注意力架构，结合压缩稀疏注意力（Compressed Sparse Attention, CSA）和高度压缩注意力（Heavily Compressed Attention, HCA），以提高长上下文效率；（2）流形约束超连接（Manifold-Constrained Hyper-Connections, mHC），增强传统残差连接；（3）以及Muon优化器，实现更快的收敛速度和更高的训练稳定性。我们在超过32T的多样化和高质量标记上对这两个模型进行了预训练，随后进行了全面的后训练流程，以解锁并进一步提升其能力。DeepSeek-V4-Pro-Max是DeepSeek-V4-Pro的最大推理努力模式，重新定义了开放模型的最先进水平，在核心任务上超越了其前身。同时，DeepSeek-V4系列在长上下文场景中表现出色。在一百万个标记的上下文设置中，与DeepSeek-V3.2相比，DeepSeek-V4-Pro仅需单标记推理浮点运算（FLOPs）的27%和键值（Key-Value, KV）缓存的10%。这使我们能够常规支持一百万个标记的上下文，从而使长时任务和进一步的测试时扩展更加可行。模型检查点可在<https://huggingface.co/collections/deepseek-ai/deepseek-v4>获取。

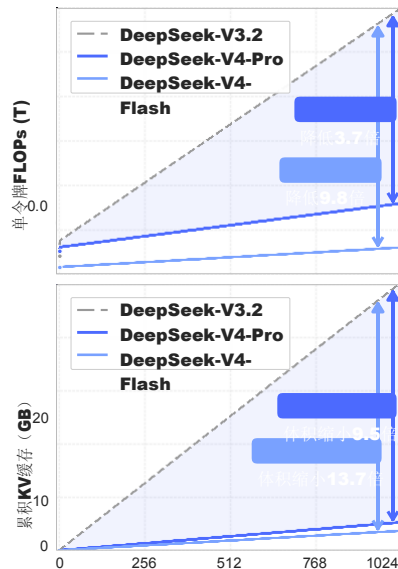
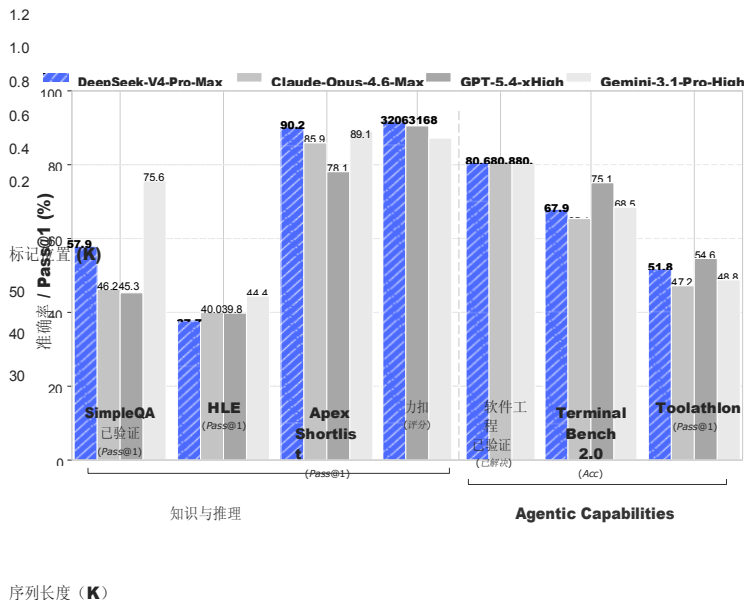


图1 | 左：DeepSeek-V4-Pro-Max及其对应产品的基准性能。右：DeepSeek-V4系列和DeepSeek-V3.2的推理FLOPs和KV缓存大小。

目录

1 引言 4.

2. 架构 6

2.1 从 DeepSeek-V3 继承的设计 7.

2.2 流形约束的超连接 7.

2.3 基于 CSA 和 HCA 的混合注意力机制. 9
. 9

2.3.1 压缩稀疏注意. 9

2.3.2 重压缩注意力. 11

2.3.3 其他细节.
. 12

2.3.4 效率讨论. 13

2.4 介子优化器.
. 14

3 通用基础设施 15

3.1 专家并行中的细粒度通信-计算重叠. . . . 15

3.2 使用 TileLang 实现灵活高效的核开发. 16

3.3 高性能批处理不变且确定性的内核库. 18

3.4 FP4 量化感知训练. 19

3.5 训练框架. 20

3.5.1 介子效率的实现.	20
3.5.2 <i>mHC</i> 的成本效益和内存效率实现.	21
3.5.3 长上下文注意力中的上下文并行性.	21
3.5.4 扩展自动微分以实现灵活激活检查点	21
3.6 推理框架.	22
3.6.1 KV 缓存结构与管理.	22
3.6.2 磁盘 KV 缓存存储.	23
4 预训练	24
4.1 数据构建.	24
4.2 预训练设置.	25
4.2.1 模型设置.	25
4.2.2 训练设置.	25
4.2.3 缓解训练不稳定性.	26
4.3 评估.	27
4.3.1 评估基准.	27
4.3.2 评估结果.	28
5 后期训练	29

5.1 后期训练流程.	29
5.1.1 专科培训.	29
5.1.2 在策略上蒸馏.	32
5.2 强化学习 (RL) 和机会决策 (OPD) 基础设施.	34
5.2.1 FP4 量化集成.	34
5.2.2 全词汇 OPD 的高效教师排课.	34
5.2.3 可抢占且容错的部署服务.	34
5.2.4 针对百万级词元上下文的扩展强化学习框架.	35
5.2.5 面向自主人工智能的沙盒基础设施.	35
5.3 标准基准评估.	36
5.3.1 评估设置.	36
5.3.2 评估结果.	38
5.4 真实世界任务上的表现.	41
5.4.1 中文写作.	41
5.4.2 搜索.	42
5.4.3 白领任务.	

42

5.4.4 代码代理.	
. . .	44

6 结论、局限性和未来方向 44

A 作者列表和致谢 54

A.1 作者列表.	
. . .	54

A.2 致谢.	55
-----------------	----

B 评估详情 55

1. 引言

推理模型 (DeepSeek-AI, 2025; OpenAI, 2024c) 的出现建立了一种新的测试时扩展范式, 为大型语言模型 (LLMs) 带来了显著的性能提升。然而, 这种扩展范式从根本上受到基础注意力机制 (Vaswani et al., 2017) 二次计算复杂性的限制, 这为超长上下文和推理过程造成了难以逾越的瓶颈。同时, 从复杂的代理工作流程到大规模跨文档分析, 长时场景和任务的出现也使得对超长上下文的有效支持成为未来进步的关键。尽管最近的开源工作 (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) 提升了通用能力, 但处理超长序列时核心架构的低效性仍是关键障碍, 限制了测试时扩展的进一步收益, 并阻碍了对长时场景和任务的进一步探索。

为了突破超长上下文中的效率瓶颈, 我们开发了 DeepSeek-V4 系列, 包括具有 1.6 万亿参数 (激活 490 亿) 的 DeepSeek-V4-Pro 预览版和具有 2840 亿参数 (激活 130 亿) 的 DeepSeek-V4-Flash 预览版。通过架构创新, DeepSeek-V4 系列在处理超长序列时实现了计算效率的显著提升。这一突破使得我们能够高效支持百万级上下文长度, 为下一代大型语言模型 (LLM) 迎来了百万级上下文的新时代。我们相信, 我们高效处理超长序列的能力开启了测试时扩展的新篇章, 为深入研究长时任务铺平了道路, 并为探索在线学习等未来范式奠定了必要的基础。

与 DeepSeek-V3 架构 (DeepSeek-AI, 2024) 相比, DeepSeek-V4 系列保留了 DeepSeekMoE 框架 (Dai 等, 2024) 和多词元预测 (MTP) 策略, 同时在架构和优化方面引入了几项关键创新。为提高长上下文效率, 我们设计了一种混合注意力机制, 该机制结合了压缩稀疏注意力 (CSA) 和高度压缩注意力 (HCA)。CSA 沿序列维度压缩键值 (KV) 缓存, 然后执行 DeepSeek 稀疏注意力 (DSA) (DeepSeekAI, 2025), 而 HCA 对 KV 缓存进行更激进的压缩, 但保持稠密注意力。为增强建模能力, 我们引入了流形约束超连接 (mHC) (Xie 等, 2026), 对传统残差连接进行了升级。此外, 我们在 DeepSeek-V4 系列的训练中引入了 Muon (Jordan 等, 2024; Liu 等, 2025) 优化器, 从而加快了收敛速度并提高了训练稳定性。

为了实现 DeepSeek-V4 系列的高效训练和推理以及高效开发，我们引入了几项基础设施优化。首先，我们为混合专家（Mixture of Experts, MoE）模块设计和实现了一个单一融合内核，该内核完全覆盖了计算、通信和内存访问。其次，我们采用了领域特定语言（Domain-Specific Language, DSL）TileLang（Wang 等人，2026），以平衡开发效率和运行时效率。第三，我们提供了高效的批量不变且确定性的内核库，以确保训练和推理过程中的逐位可复现性。第四，我们为 MoE 专家权重和索引器 QK 路径引入了 FP4 量化感知训练，以减少内存和计算量。第五，在训练框架方面，我们扩展了自动微分框架，增加了张量级检查点功能，以实现细粒度的重计算控制；我们还通过混合式零重新计算（ZeRO）策略、通过重计算和融合内核实现的成本效益高的混合硬件（mixed hardware, *mHC*）以及两阶段上下文并行性来管理压缩注意力，从而提高了训练效率。最后，在推理框架方面，我们设计了一个具有磁盘存储策略的异构键值（Key-Value, KV）缓存结构，以实现高效的共享前缀重用。

通过采用混合计算图（CSA）和混合注意力图（HCA），以及对计算和存储进行精确优化，DeepSeek-V4 系列相较于 DeepSeek-V3.2 实现了显著更低的推理浮点运算次数（FLOPs）和大幅缩减的键值（KV）缓存大小，尤其是在长上下文设置中。图 1 右侧展示了 DeepSeek-V3.2 和 DeepSeek-V4 系列的单词推理 FLOPs 估算值和累积 KV 缓存大小。在 1 百万词上下文设置中，即使激活参数数量更多的 DeepSeek-V4-Pro，其单词 FLOPs 也仅为 DeepSeek-V3.2 的 27%（以等效的 FP8 FLOPs 衡量），KV 缓存大小仅为 10%。此外，激活参数数量更少的 DeepSeek-V4-Flash 进一步提升了效率：在 1 百万词上下文设置中，其单词 FLOPs 和 KV 缓存大小分别仅为 DeepSeek-V3.2 的 10% 和 7%。另外，对于 DeepSeek-V4 系列，路由专家参数采用 FP4 精度。虽然目前 $\text{FP4} \times \text{FP8}$ 运算的峰值 FLOPs 与现有硬件上的 $\text{FP8} \times \text{FP8}$ 相同，但理论上在未来硬件上可以实现 $1/3$ 的效率提升，这将进一步提高 DeepSeek-V4 系列的效率。

在预训练阶段，我们分别使用 32T 个标记训练 DeepSeek-V4-Flash，使用 33T 个标记训练 DeepSeek-V4-Pro。预训练后，这两个模型能够原生且高

效地支持 1M 长度的上下文。在我们的内部评估中，凭借其更高效的参数设计，DeepSeek-V4-Flash-Base 在大多数基准测试中已经超越了 DeepSeek-V3.2-Base。DeepSeek-V4-Pro-Base 进一步扩大了这一优势，在 DeepSeek 基础模型中树立了新的性能标准，在推理、编码、长上下文和世界知识任务中均表现出全面优势。

DeepSeek-V4 系列的后训练流程采用两阶段范式：首先独立培养领域特定的专家模型，然后通过策略内蒸馏进行统一模型整合 (Lu and Lab, 2025)。最初，对于每个目标领域（如数学、编程、代理和指令遵循），会独立训练一个单独的专家模型。基础模型首先在高质量的领域特定数据上进行监督微调 (SFT)，以建立基础能力。随后，使用组相对策略优化 (GRPO) (DeepSeek-AI, 2025) 应用强化学习 (RL)，该方法在针对特定成功标准定制的奖励模型的指导下，进一步优化模型以实现与领域一致的行为。这一阶段会产生一组多样化的专业专家，每个专家在其各自领域表现出色。最后，为了整合这些不同的专长，通过策略内蒸馏训练一个统一的模型，其中统一模型作为学生模型，学习优化与教师模型的逆向 KL 损失。

核心评估结果总结

- **知识：**在广泛世界知识的评估中，DeepSeek-V4-Pro-Max (DeepSeek-V4-Pro 的最大推理努力模式) 在 SimpleQA (OpenAI, 2024d) 和 Chinese-SimpleQA (He 等人, 2024) 基准测试上显著优于领先的开源模型。在教育知识方面——通过 MMLU-Pro (Wang 等人, 2024b)、HLE (Phan 等人, 2025) 和 GPQA (Rein 等人, 2023) 进行评估——DeepSeek-V4-Pro-Max 相较于其开源对应模型表现出微弱优势。尽管在这些基于知识的评估中，DeepSeek-V4-Pro-Max 仍然落后于领先的专有模型 Gemini-3.1-Pro，但它已显著缩小了与该模型的差距。
- **推理：**通过扩展推理标记，DeepSeek-V4-Pro-Max 在标准推理基准测试中表现出优于 GPT-5.2 和 Gemini-3.0-Pro 的性能。然而，其性能略逊于 GPT-5.4 和 Gemini3.1-Pro，表明其发展轨迹落后于最先进的前沿模型约

3 至 6 个月。此外，DeepSeek-V4-Flash-Max 的性能与之相当

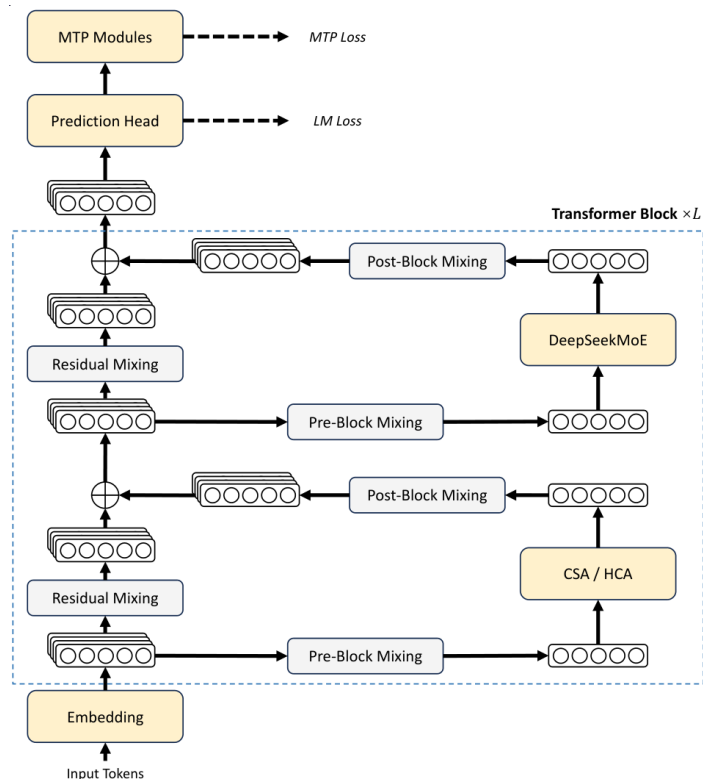


图 2 | DeepSeek-V4 系列的总体架构。我们在注意力层使用混合压缩稀疏注意力 (Compressed Sparse Attention, CSA) 和高度压缩注意力 (Heavily Compressed Attention, HCA)，在前馈层使用 DeepSeekMoE，并通过 *mHC* 加强了传统的残差连接。

其性能与 GPT-5.2 和 Gemini-3.0-Pro 相当，从而确立了其作为复杂推理任务中极具成本效益的架构的地位。

- **代理：**在公开基准测试中，DeepSeek-V4-Pro-Max 与领先的开源模型（如 Kimi-K2.6 和 GLM-5.1）相当，但略逊于前沿的封闭模型。在我们的内部评估中，DeepSeek-V4-Pro-Max 的表现优于 Claude Sonnet 4.5，并接近 Opus 4.5 的水平。
- **长上下文：**DeepSeek-V4-Pro-Max 在合成和真实用例中表现强劲，其上下文窗口可达 100 万个标记，在学术基准测试中甚至超越了 Gemini-3.1-Pro。
- **DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 的比较：**由于参数规模较小，DeepSeek-V4-Flash-Max 在知识评估中的表现较差。然而，当分配到更大的思考预算时，它在推理任务上取得了与前者相当的结果。在

代理评估中，虽然 DeepSeek-V4-Flash-Max 在多个基准测试上的表现与 DeepSeek-V4-Pro-Max 相当，但在更复杂、难度更高的任务上，它仍然落后于后者。

2. 架构

总体而言，DeepSeek-V4 系列保留了 Transformer (Vaswani 等人, 2017) 架构和多词预测 (MTP) 模块 (DeepSeek-AI, 2024; Gloeckle 等人, 2024)，同时相对于 DeepSeek-V3 引入了几项关键升级：(1) 首先，我们引入了流形约束超连接 (mHC) (Xie 等人, 2026) 以加强传统的残差连接；(2) 其次，我们设计了一种混合注意力架构，通过压缩稀疏注意力和高度压缩注意力显著提高了长上下文效率。(3) 第三，我们采用 Muon (Jordan 等人, 2024 年; Liu 等人, 2025 年) 作为优化器。对于混合专家 (Mixture-of-Experts, MoE) 组件，我们仍然采用 DeepSeekMoE (Dai 等人, 2024 年) 架构，仅对 DeepSeek-V3 进行了微调。多标记预测 (Multi-Token Prediction, MTP) (DeepSeek-AI, 2024 年; Gloeckle 等人, 2024 年; Li 等人, 2024 年; Qi 等人, 2020 年) 配置与 DeepSeek-V3 保持一致。所有其他未指定的细节均遵循 DeepSeekV3 (DeepSeek-AI, 2024 年) 中已建立的设置。图 2 展示了 DeepSeek-V4 的整体架构，详细内容如下所述。

2.1. 继承自 DeepSeek-V3 的设计

混合专家。与之前的 DeepSeek 系列模型 (DeepSeek-AI, 2024; DeepSeek-AI, 2024) 一样，DeepSeek-V4 系列也采用了 DeepSeekMoE 范式 (Dai et al., 2024) 用于前馈网络 (FFNs)，该范式设置了细粒度的路由专家和共享专家。与 DeepSeek-V3 不同，我们将计算亲和度得分的激活函数从 $\text{Sigmoid}(\cdot)$ 更改为 $\text{Sqrt}(\text{Softplus}(\cdot))$ 。为了实现负载均衡，我们还采用了无辅助损失策略 (DeepSeek-AI, 2024; Wang et al., 2024a)，并辅以轻微的序列级平衡损失，以防止单个序列内部出现极端不平衡的情况。对于 DeepSeek-V4，我们取消了对路由目标节点数量的限制，并仔细重新设计了并行策略以保持训练

效率。此外，与 DeepSeek-V3 相比，我们将初始的几个 Transformer 块中的密集 FFN 层替换为采用哈希路由（Roller et al., 2021）的 MoE 层。哈希路由策略根据输入标记 ID 的预定义哈希函数确定每个标记的目标专家。

多模态词元预测。与 DeepSeek-V3 一样，DeepSeek-V4 系列也设置了多模态词元预测（MTP）模块和目标。鉴于 MTP 策略已在 DeepSeek-V3 中得到验证，我们未做修改，直接将其应用于 DeepSeek-V4 系列。

2.2. 流形约束超连接

如图 2 所示，DeepSeek-V4 系列采用了流形约束超连接（ mHC ）（Xie 等人，2026）来加强相邻 Transformer 块之间的传统残差连接。与朴素超连接（HC）（Zhu 等人，2025）相比， mHC 的核心思想是将残差映射约束到特定的流形上，从而在保持模型表现力的同时增强信号跨层传播的稳定性。本小节简要介绍了标准 HC，并描述了我们如何设计 mHC 以实现稳定训练。

标准超连接。标准超连接（HC）将残差流的宽度扩展了 N_{hc} 倍。具体而言，残差流的形状从 $\mathbf{R}^D$ 扩展到 $\mathbf{R}^{N_{hc} \times D}$ ，其中 D 是实际层输入的隐藏层大小。设 $X_L = [\mathbf{x}_{L,1}; \dots; \mathbf{x}_{L,n_{hc}}]^T \in \mathbb{R}^{n_{hc} \times d}$ 为第 L 层之前的残差状态。超连接引入了三个线性映射：输入映射 $A_L \in \mathbb{R}^{1 \times n_{hc}}$ 、残差变换 $B_L \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 和输出映射 $C_L \in \mathbb{R}^{n_{hc} \times 1}$ 。然后，残差状态的更新公式为：

$$X_{L+1} = B_L X_L + C_L \mathcal{F}_L(A_L X_L), \quad (1)$$

其中， L 表示第 L 层（例如，一个混合专家（Mixture of Experts, MoE）层），其输入和输出形状均为 $\mathbf{R}^{N_{hc} \times D}$ 。请注意，实际的层输入 $A_L X_L \in \mathbb{R}^d$ 也是 D 维的，因此扩展的残差宽度不会影响内部层的设计。混合残差通道（Hybrid Channel, HC）将残差宽度与实际的隐藏单元大小解耦，以最小的计算开销提供了一个互补的缩放轴，因为 n_{hc} 通常远小于隐藏单元大小 D 。然而，尽管 HC 在提高模型性能方面表现出潜力，我们发现当堆叠多层时，训练会频繁

出现数值不稳定的情况，这阻碍了 HC 的缩放。

流形约束残差映射。 m HC 的核心创新是将残差映射矩阵 B_L 约束到双随机矩阵的流形（即 Birkhoff 多面体） M 上，从而增强信号在层间传播的稳定性：

$$B_L \in \mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M\mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\}. \quad (2)$$

这一约束确保映射矩阵 $\|B_L\|_2$ 的谱范数有界于 1，从而使残差变换具有非扩张性，这提高了前向传播和反向传播过程中的数值稳定性。此外，集合 M 在乘法下是封闭的，这保证了在多层堆叠 m HC 场景下的稳定性。另外，输入变换 A_L 和输出变换 C_L 也通过 Sigmoid 函数被约束为非负且有界，以避免信号抵消的风险。

动态参数化。 三个线性映射的参数是动态生成的，它们被分解为动态（输入依赖）组件和静态（输入独立）组件。给定输入 $X_l \in \mathbb{R}^{n_{hc} \times d}$ ，首先对其进行展平和归一化处理： $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{hc}d}$ 。然后，我们遵循传统的超参数控制（Hyperparameter Control, HC）方法来生成无约束的原始参数 $\tilde{A}_l \in \mathbb{R}^{1 \times n_{hc}}$, $\tilde{B}_l \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 和 $\tilde{C}_l \in \mathbb{R}^{n_{hc} \times 1}$ ：

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \quad (3)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \quad (4)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \quad (5)$$

其中 $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{hc}d \times n_{hc}}$ 和 $W_l^{\text{res}} \in \mathbb{R}^{n_{hc}d \times n_{hc}^2}$ 是用于生成动态组件的可学习参数； $\text{Mat}(\cdot)$ 将大小为 $1 \times n_{hc}^2$ 的向量重塑为大小为 $n_{hc} \times n_{hc}$ ； $S_l^{\text{pre}} \in \mathbb{R}^{1 \times n_{hc}}$, $S_l^{\text{post}} \in \mathbb{R}^{n_{hc} \times 1}$ 的矩阵， $S_l^{\text{res}} \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 是可学习的静态偏差； $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ 是初始化为小值的可学习门控因子。

应用参数约束。 在获得无约束的原始参数 $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$ 后，我们对其应用之前描述的约束，以提高数值稳定性。具体而言，对于输入和输出映射，我们采用

Sigmoid 函数 $(\cdot)$ 来确保其非负性和有界性：

$$A_l = \sigma(\tilde{A}_l), \quad (6)$$

$$C_l = 2\sigma(\tilde{C}_l). \quad (7)$$

至于残差映射 $\tilde{B}_l$ ，我们将其投影到双随机矩阵 M 的流形上。这是通过 Sinkhorn-Knopp 算法实现的，该算法首先对 $\tilde{B}_l$ 应用指数函数以确保正性，得到 $M^{(0)} = \exp(\tilde{B}_l)$ ，然后迭代执行列和行归一化：

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \quad (8)$$

其中 τ_r 和 τ_c 分别表示行归一化和列归一化。此迭代收敛于一个受约束的双随机矩阵 $B_l = M^{(t_{\max})}$ 。我们选择 $t_{\max} = 20$ 作为实际值。

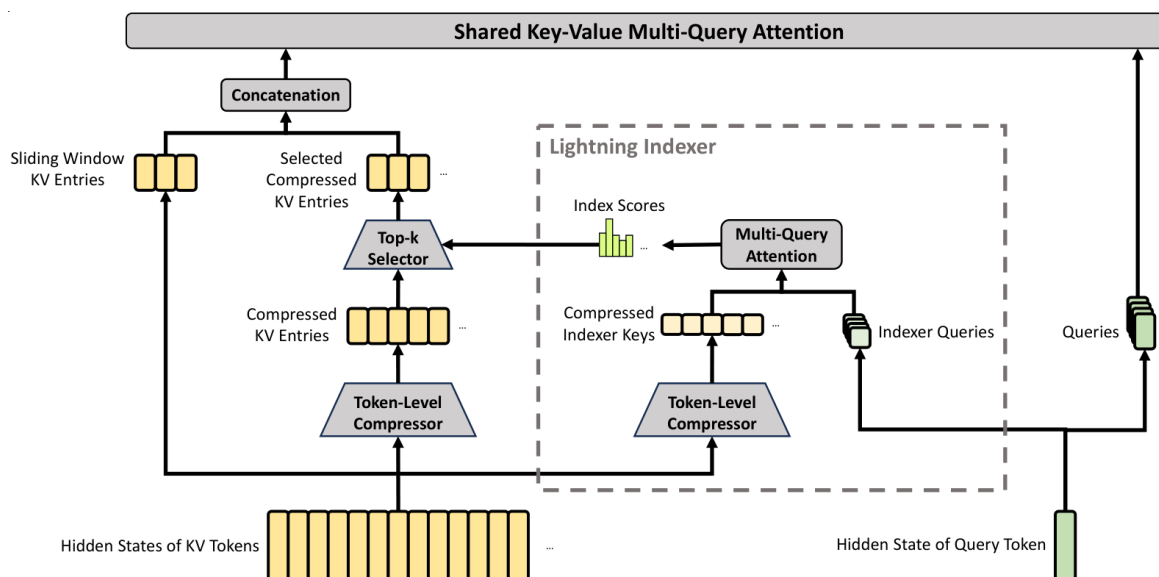


图 3 | 压缩自注意力（CSA）的核心架构。它将键值（KV）条目的数量压缩至 $\frac{1}{M}$ 倍，然后应用 DeepSeek 稀疏注意力以进一步加速。此外，将一小部分滑动窗口 KV 条目与所选压缩 KV 条目相结合，以增强局部细粒度依赖关系。

2.3. 结合 CSA 和 HCA 的混合注意力

当上下文长度达到极端规模时，注意力机制成为模型中的主要计算瓶颈。对于 DeepSeek-V4，我们设计了两种高效的注意力架构——压缩稀疏注意力（CSA）和高度压缩注意力（HCA）——并采用了它们的交错混合配置，这大大降低了长文本场景中注意力的计算成本。CSA 融合了压缩和稀疏注意力策略：它首先将每个 M 个标记的键值（KV）缓存压缩为一个条目，然后

应用 DeepSeek 稀疏注意力 (DSA) (DeepSeek-AI, 2025), 其中每个查询标记仅关注 K 个压缩后的 KV 条目。HCA 旨在通过将每个 $M'(\gg M)$ 个标记的 KV 缓存合并为一个条目来实现极端压缩。CSA 和 HCA 的混合架构显著提高了 DeepSeek-V4 系列的长上下文效率, 使得处理一百万个标记的上下文在实践中成为可能。本小节描述了我们混合注意力架构的核心技术, 并提供了一个开源实现¹, 以明确地说明更多细节。

2.3.1. 压缩稀疏注意

CSA 的核心架构如图 3 所示, 该架构首先将每个 M 个 token 的 KV 缓存压缩为一个条目, 然后应用 DeepSeek 稀疏注意力以进一步加速。

压缩键值对。 设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态的序列, 其中 N 为序列长度, D 为隐藏单元大小。CSA 首先计算两组键值对 $C^a, C^b \in \mathbb{R}^{\tilde{n} \times c}$ 及其对应的压缩权重 $Z^a, Z^b \in \mathbb{R}^{n \times c}$, 其中 C 为头部维度:

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \quad (9)$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \quad (10)$$

其中 $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来, 根据压缩权重和可学习的位置偏差 $B^a, B^b \in \mathbb{R}^{m \times c}$, 将 C^a 和 C^b 中的每个 M 个 KV 条目压缩为一个条目, 得到 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m} \times c}$ 。每个压缩后的条目 $c_i^{\text{Comp}} \in \mathbb{R}^c$ 是通过以下公式计算的:

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]), \quad (11)$$

$$c_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \quad (12)$$

其中, $\odot$ 表示哈达玛积; $\text{Softmax}_{\text{row}}(\cdot)$ 表示沿行维度的 softmax 操作, 该操作对来自 Z^a 和 Z^b 的总共 $2M$ 个元素进行归一化。此时, $i=0, Z_{m(i-1):mi-1}^b$ 用负无穷填充, $C_{m(i-1):mi-1}^b$ 用零填充。请注意, 每个 c_i^{Comp} 都源自 $2M$ 个键值 (KV) 条目,

¹<https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference>

但用于 c_i^{Comp} 的 c^b 索引和用于 c_{i-1}^{Comp} 的 c^a 索引是重叠的。因此，CSA 实际上将序列长度压缩到了 $\frac{1}{m}$ 倍。

稀疏选择用闪电索引器。在获取压缩后的键值 (KV) 条目 C^{Comp} 后，压缩自注意力 (CSA) 采用动态自注意力 (DSA) 策略来选择前 k 个压缩后的 KV 条目以进行核心注意力计算。首先，CSA 执行与 C^{Comp} 相同的压缩操作，以获得压缩后的索引器键 $K^{\text{IComp}} \in \mathbb{R}^{\frac{n}{m} \times c^I}$ ，其中 C^I 是索引器头维度。然后，对于查询标记 T ，我们以低秩方式生成索引器查询 $\{\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_t^I}^I\}$:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (13)$$

$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_t^I}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ}, \quad (14)$$

其中 F_{77} 是查询标记 T 的输入隐藏状态; F_{78} 是查询的压缩潜在向量; d_c 表示查询压缩维度; n_h^I 表示索引器查询头的数量; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n_h^I}$ 分别是索引器查询的下投影矩阵和上投影矩阵。接下来，通过以下公式计算查询标记 T 与前一个压缩块 S ($s < \text{Floor}(\frac{1}{m})$) 之间的索引分数 F_{83} :

$[W^I T, 1; W^I T, 2; \dots; W^I T, NI] = \mathbf{w} I_T = \mathbf{h} T \cdot W^W$, (15) 其中 $W^W \in \mathbb{R}^{d \times n_h^I}$ 是一个可学习的矩阵; $w_{t,h}^I \in \mathbb{R}$ 是第 h 个索引头的权重。对于查询标记 T ，给定其索引分数 $I_{t,:}$ ，我们采用 top-k 选择器来有选择地保留压缩后的 KV 条目 C_t^{SprsComp} 的子集，以用于后续的核心注意力机制:

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}), \quad (16)$$

$$C_t^{\text{SprsComp}} = \left\{ C_s^{\text{Comp}} \mid I_{t,s} \in \text{Top-k}(I_{t,:}) \right\}. \quad (17)$$

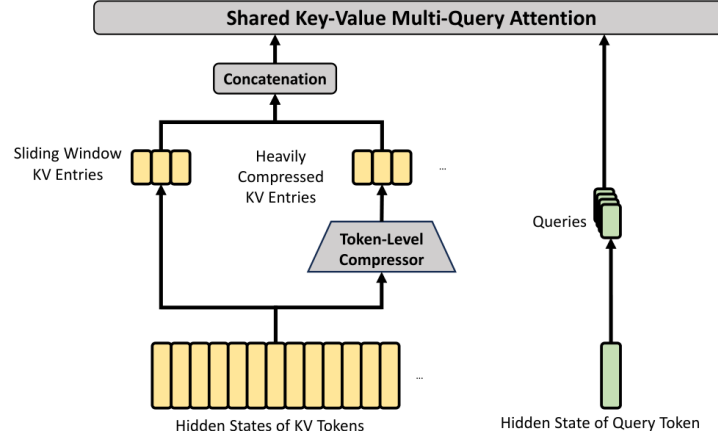


图 4 | HCA 的核心架构。它执行更重的压缩，其中 $M/(\gg M)$ 令牌的 KV 条目将被合并为一个。此外，我们还额外引入了一小部分滑动窗口 KV 条目，以增强局部细粒度依赖性。

共享键值多查询注意力 (MQA)。在选择了稀疏的键值 (KV) 条目后，压缩自注意力 (CSA) 以多查询注意力 (MQA) (Shazeer, 2019) 的方式执行核心注意力，其中 c_t^{SprsComp} 中的每个压缩键值条目同时作为注意力键和值。具体而言，对于查询标记 T ，我们首先从压缩潜在向量 c_t^Q 中生成注意力查询 $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ ：

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (18)$$

其中， n_h 表示查询头的数量； $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ 是查询的上投影矩阵。请注意，潜在查询向量 c_t^Q 与用于索引查询的向量是共享的。接下来，我们对 $\{\mathbf{q}_{t,i}\}$ 和 c_t^{SprsComp} 执行 MQA 操作：

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=\mathbf{c}_t^{\text{SprsComp}}, \text{value}=\mathbf{c}_t^{\text{SprsComp}}), \quad (19)$$

其中， $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 表示第 T 个标记处第 I 个头的核心注意力输出； $\text{CoreAttn}(\cdot)$ 表示核心注意力操作。

分组输出投影。在 DeepSeek-V4 的配置中， CN 相当大。因此，直接将核心注意力操作 $[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] = \mathbf{o}_t \in \mathbb{R}^{cn_h}$ 的输出投影到 D 维隐藏状态将带来巨大的计算负担。为了降低这一成本，我们设计了一种分组输出投影策略。具体来说，

我们首先将 n_h 输出分为 G 组，然后对于每一组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$ ，将其投影到 d_g 维的中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c \frac{n_h}{g}$ 。最后，我们将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影到最终的注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.2. 重压缩注意力机制

重压缩注意力机制（Heavily Compressed Attention, HCA）的核心架构如图 4 所示，该架构以更重的方式压缩 KV 缓存，但并未采用稀疏注意力机制。

压缩键值条目。 总体而言，HCA 的压缩策略与 CSA 相似，但采用了更大的压缩率 $M' (\gg M)$ ，并且不执行重叠压缩。设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态的序列，HCA 首先计算原始键值（Key-Value, KV）条目 $C \in \mathbb{R}^{n \times c}$ 及其相应的压缩权重 $Z \in \mathbb{R}^{n \times c}$ ：

$$C = H \cdot W^{KV}, \quad (20)$$

$$Z = H \cdot W^Z, \quad (21)$$

其中 $W^{KV}, W^Z \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来，根据压缩权重和可学习的位置偏差 $B \in \mathbb{R}^{m' \times c}$ ，将 C 中的每个 M' 个 KV 条目压缩成一个，得到 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$ 。每个压缩后的条目 $c_i^{\text{Comp}} \in \mathbb{R}^c$ 是通过以下公式计算的：

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \quad (22)$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \quad (23)$$

通过此次压缩操作，HCA 将序列长度压缩至 $\frac{1}{m'}$ 倍。

共享键值多查询聚合（MQA）与分组输出投影。 HCA 也采用了与 CSA 相同的共享键值多查询聚合（MQA）和分组输出投影策略。在键值压缩后，对于查询标记 T ，HCA 首先以低秩方式生成注意力查询 $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ ：

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (24)$$

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

其中 $\mathbf{h}_t \in \mathbb{R}^d$ 是查询标记 T 的输入隐藏状态; N 表示查询头的数量; $W^{DQ} \in \mathbb{R}^{d \times d}$ 和 $W^{UQ} \in \mathbb{R}^{d_c \times d}$ 分别是查询的下投影矩阵和上投影矩阵。接下来, 我们对 $\{\mathbf{q}_{t,i}\}$ 和 C^{Comp} 执行多头查询注意力 (MQA) 操作:

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=\mathbf{C}^{\text{Comp}}, \text{value}=\mathbf{C}^{\text{Comp}}), \quad (26)$$

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 T 个标记处第 I 个头的核心注意力输出。接下来, 与 CSA 一样, HCA 将 N 个输出分为 G 组, 对于每一组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$, HCA 将其投影到一个 d_g 维的中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$, 其中 $d_g < c \frac{n_h}{g}$ 。最后, HCA 将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影到最终的注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.3. 其他细节

除了上述的 CSA 和 HCA 核心架构外, 我们的混合注意力还融合了其他多种技术。为了使表述更加清晰, 我们在上述介绍中省略了这些额外技术, 并将在本小节中简要介绍。此外, 本小节仅关注这些技术的核心思想, 为简洁起见, 可能会省略一些细微的细节。我们鼓励读者查阅我们的开源实现以获取明确的细节。

查询与键值条目归一化。对于 CSA 和 HCA, 我们在核心注意力操作之前, 对查询的每个头以及压缩键值 (KV) 条目的唯一头执行额外的 RMSNorm 操作。这种归一化可避免注意力逻辑值爆炸, 并可能提高训练稳定性。

部分旋转位置嵌入。对于循环自注意力 (CSA) 和分层循环自注意力 (HCA), 我们部分地将旋转位置嵌入 (RoPE) (Su 等人, 2024) 应用于注意力查询、键值对 (KV) 条目和核心注意力输出。具体而言, 对于在 CSA 和 HCA 中使用的每个查询向量和 KV 条目向量, 我们对其最后 64 个维度应用 RoPE。由于 KV 条目同时作为注意力键和值, 因此朴素的核心注意力输出 $\{\mathbf{o}_{t,i}\}$ 将包含绝对位置嵌入, 该嵌入由 KV 条目的加权和得出。作为对策, 我们还在每个 $\mathbf{o}_{t,i}$ 的最后 64 个维度上应用带有位置 $-I$ 的 RoPE。这样, 核心注意力的输出也将包含相对位置嵌入——每个 KV 条目对核心注意力输出的贡献也将

与查询和 KV 条目之间的距离相关。

滑动窗口注意力机制的附加分支。为了在压缩自注意力（CSA）和压缩全注意力（HCA）中严格保持因果关系，每个查询仅关注其前方的压缩键值（KV）块。因此，查询无法访问其自身压缩块内其他标记的信息。同时，在语言建模中，最近的标记通常与查询标记具有更大的相关性。出于这些原因，我们以滑动窗口的方式为 CSA 和 HCA 引入了一个补充注意力分支，以更好地建模局部依赖关系。具体而言，对于每个查询标记，我们额外生成 N 个未压缩的 KV 条目，这些条目对应于最近的 n_{win} 个标记。在 CSA 和 HCA 的核心注意力机制中，这些滑动窗口中的 KV 条目将与压缩的 KV 条目一起使用。

注意力汇聚。在 CSA 和 HCA 的核心注意力机制中，我们采用了注意力汇聚的技巧（OpenAI, 2025; Xiao 等人, 2024）。具体来说，我们设置了一系列可学习的汇聚逻辑值 $\{z'_1, z'_2, \dots, z'_{n_h}\}$ 。对于第 i 个注意力头， $\text{Exp}(Z_i)$ 将被添加到注意力得分的分母中：

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \quad (27)$$

其中 $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ 表示第 i 个查询词元与第 j 个前导词元或压缩块之间第 i 个注意力头的注意力得分和注意力逻辑值。该技术允许每个查询头调整其总注意力得分，使其不等于 1，甚至接近 0。

2.3.4. 效率讨论

由于采用了混合 CSA 和 HCA，以及低精度计算和存储，DeepSeek-V4 系列的注意力模块在注意力浮点运算次数（FLOPs）和键值（KV）缓存大小方面均实现了显著效率提升，尤其是在长上下文场景中。首先，我们为 KV 条目采用了一种混合存储格式：旋转位置嵌入（RoPE）维度使用 BF16 精度，而其余维度则采用 FP8 精度。与纯 BF16 存储相比，这种混合表示将 KV

缓存大小减少了近一半。其次，闪电索引器内的注意力计算以 FP4 精度进行，这加速了极长上下文下的注意力操作。第三，与 DeepSeek-V3.2 相比，DeepSeek-V4 系列选择了更小的注意力 top-k，从而提高了模型在中短文本上的效率。最后，也是最重要的是，压缩注意力和混合注意力技术大幅减少了 KV 缓存大小和计算浮点运算次数（FLOPs）。

以头部维度为 128 的 BF16 GQA8（Ainslie 等人，2023）作为基线——这是大型语言模型（LLM）注意力机制的常见配置之一——在 1M 上下文设置下，DeepSeek-V4 系列的键值（KV）缓存大小可大幅降低至基线大小的约 2%。

Algorithm 1 Muon Optimizer for DeepSeek-V4

Require: Learning rate η , momentum μ , weight decay λ , update rescaling factor γ

```

1: for each training step  $t$  do
2:   for each logically independent weight  $W \in \mathbb{R}^{n \times m}$  do
3:      $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$  ▷ Compute gradients
4:      $M_t = \mu M_{t-1} + G_t$  ▷ Accumulate momentum buffer
5:      $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$  ▷ Nesterov trick and hybrid Newton-Schulz
6:      $O_t = O'_t \cdot \sqrt{\max(n, m)} \cdot \gamma$  ▷ Rescale the update RMS
7:      $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$  ▷ Perform weight decay and update
8:   end for
9: end for
```

此外，即使与 DeepSeek-V3.2（DeepSeek-AI，2025）——这已经是一个高效的基线模型——相比，DeepSeek-V4 系列在效率上仍然具有显著优势。图 1 右侧部分提供了它们的推理浮点运算次数（FLOPs）和键值（KV）缓存大小的对比。

2.4. Muon Optimizer

由于 Muon（Jordan 等人，2024 年；Liu 等人，2025 年）优化器具有更快的收敛速度和更高的训练稳定性，我们在 DeepSeek-V4 系列的大多数模块中采用了该优化器。我们的 Muon 优化的完整算法在算法 1 中进行了总结。

基本配置。我们为嵌入模块、预测头模块、 m HC 模块的静态偏差和门控因子

以及所有 RMSNorm 模块的权重使用 AdamW (Loshchilov 和 Hutter, 2017) 优化器。所有其他模块均使用 Muon 进行更新。遵循 Liu 等人 (2025) 的做法, 我们还对 Muon 参数应用权重衰减, 使用 Nesterov (Jordan 等人, 2024; Nesterov, 1983) 技巧, 并重新缩放更新矩阵的均方根 (RMS) 以重新利用我们的 AdamW 超参数。与他们不同的是, 我们使用混合 Newton-Schulz 迭代进行正交化。

混合牛顿-舒尔茨迭代。对于给定的矩阵 M , 设其奇异值分解 (SVD) 为 $M = U\Sigma V^T$ 。牛顿-舒尔茨迭代旨在近似正交化 M , 使其成为 UV^T 。通常, 首先将 M 归一化为 $M_0 = M/\|M\|_F$, 以确保其最大奇异值不超过 1。然后, 每次牛顿-舒尔茨迭代执行以下操作:

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \quad (28)$$

我们的混合 Newton-Schulz 算法在两个不同的阶段执行 10 次迭代。在前 8 步中, 我们使用系数 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ 来驱动快速收敛, 使奇异值接近 1。在最后 2 步中, 我们切换到系数 $(a, b, c) = (2, -1.5, 0.5)$, 该系数能将奇异值精确稳定在 1。

避免注意力逻辑值爆炸。DeepSeek-V4 系列的注意力架构允许我们直接在注意力查询和键值 (KV) 条目上应用均方根正则化 (RMSNorm), 这有效防止了注意力逻辑值爆炸。因此, 在我们的 Muon 优化器中, 我们没有采用 QK-Clip 技术 (Liu 等人, 2025)。

3. 通用基础设施

3.1. 专家并行中的细粒度通信-计算重叠

专家混合 (Mixture-of-Experts, MoE) 可通过专家并行 (Expert Parallelism, EP) 实现加速。然而, EP 需要复杂的节点间通信, 并对互连带宽和延迟提

出了较高要求。为了缓解 EP 中的通信瓶颈，并在较低的互连带宽要求下实现更高的端到端性能，我们提出了一种细粒度 EP 方案，该方案将通信和计算融合到一个流水线内核中，以实现通信-计算重叠。

通信延迟可被隐藏。我们提出的 EP 方案的关键见解是，通信延迟可有效地隐藏在 MoE 层的计算之下。如图 5 所示，在 DeepSeek-V4 系列中，每个 MoE 层主要可分解为四个阶段：两个通信绑定阶段（Dispatch 和 Combine）和两个计算绑定阶段（Linear-1 和 Linear-2）。我们的性能分析显示，在单个 MoE 层内，通信总时间少于计算时间。因此，在将通信和计算融合到统一流水线后，计算仍然是主要的瓶颈，这意味着系统可以容忍较低的互连带宽，而不会降低端到端的性能。

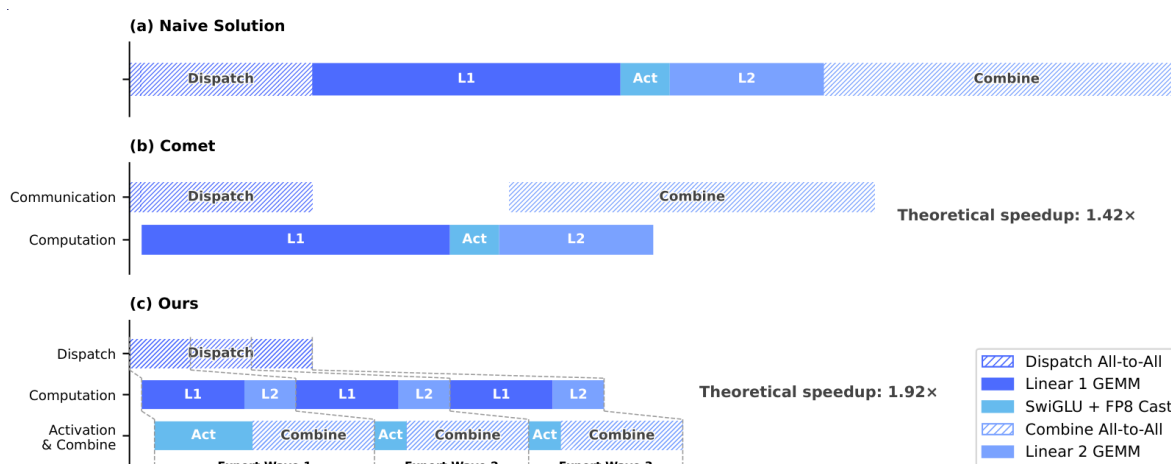


图 5 | 我们的专家并行 (EP) 方案与相关工作的对比图示。Comet (Zhang 等人, 2025b) 将 Dispatch 与 Linear-1 重叠，将 Linear-2 与 Combine 重叠。我们的 EP 方案通过将专家拆分并调度成波次，实现了更精细的交叠。理论加速比是在 DeepSeek-V4-Flash 架构的配置下评估的。

细粒度专家划分方案。为了进一步降低互连带宽需求并放大重叠的益处，我们引入了一种更细粒度的专家划分方案。受许多相关工作 (Aimuyo 等人, 2025 年; Zhang 等人, 2025b) 的启发，我们将专家拆分为多个波次并进行调度。每个波次包含一小部分专家。一旦波次内的所有专家完成通信，即可立即开始计算，无需等待其他专家。在稳态下，当前波次的计算、下一波次的令牌传递以及已完成专家的结果发送均并行进行，如图 5 所示。这形成了

专家之间的细粒度流水线，使整个波次中的计算和通信保持连续。基于波次的调度在极端情况下（如强化学习（RL）的滚动执行，通常会遇到长尾小批量）能显著提升性能。

性能与开源巨型内核。我们在 NVIDIA GPU 和华为 Ascend NPUs 平台上验证了细粒度 EP 方案。与强大的非融合基线相比，该方案在通用推理工作负载上实现了 $1.50 \sim 1.73\times$ 的速度提升，在延迟敏感场景（如 RL 展示和高速代理服务）中实现了高达 $1.96\times$ 的速度提升。我们已将基于 CUDA 的巨型内核实现命名为 **MegaMoE**²，作为 DeepGEMM 的一个组件开源。

观察与建议。我们分享了从内核开发中获得的观察与经验教训，并向硬件供应商提出了一些建议，希望有助于高效硬件设计并实现更好的软硬件协同设计：

- **计算-通信比。**完全通信-计算重叠取决于计算-通信比，而不仅仅是带宽。设峰值计算吞吐量为 C ，互连带宽为 B ，当 $C/B \leq V_{\text{comp}}/V_{\text{comm}}$ 时，通信可被完全隐藏，其中 V_{comp} 表示计算量， V_{comm} 表示通信量。对于 DeepSeek-V4-Pro，其中每个标记-专家对需要 $6D$ FLOPs（SwiGLU 门、上投影和下投影），但只需要 3 字节的通信（FP8 调度 + BF16 合并），这简化为：

$$\frac{C}{B} \leq 2d = 6144 \text{ FLOPs/Byte.}$$

即，每 GBps 的互连带宽足以隐藏 6.1 TFLOP/s 的计算通信。一旦带宽达到这一阈值，它就不再是瓶颈，而将额外的硅片面积用于增加带宽所带来的收益将递减。我们鼓励未来的硬件设计瞄准这样的平衡点，而不是无条件地扩展带宽。

- **功耗预算。**极端的内核融合会同时将计算、内存和网络推向高负载，使得功耗限制成为关键的性能瓶颈。我们建议未来的硬件设计为这种完全

²<https://github.com/deepseek-ai/DeepGEMM/pull/304>

并发的工作负载提供足够的功耗余量。

- **通信原语**。我们采用基于拉取的方法，其中每个 GPU 主动从远程 GPU 读取数据，避免了细粒度推送所带来的高通知延迟。未来硬件若具备更低延迟的跨 GPU 信号传输功能，将使推送变得可行，并支持更自然的通信模式。
- **激活函数**。我们提出用一种不涉及指数或除法运算的低成本逐元素激活函数来替换 SwiGLU。这直接减轻了 GEMM（广义矩阵乘法）后的处理负担，并且在相同的参数预算下，移除门投影操作会增大中间维度 D ，从而进一步放宽带宽要求。

3.2. 使用 TileLang 实现灵活高效的核开发

在实践中，我们精心设计的模型架构会产生数百个细粒度的 Torch ATen 运算符。我们采用 TileLang (Wang 等人, 2026) 开发了一组融合核来替换其中绝大多数运算符，以最小的努力实现了最优的性能。它还使我们能够在验证期间快速原型化注意力变体等运算符。这些核在模型架构开发、大规模训练以及最终的生产部署推理服务中发挥着关键作用。作为领域特定语言 (DSL)，TileLang 在开发效率和运行时效率之间取得了平衡，能够在支持同一代码库内深度迭代优化的同时实现快速开发。此外，我们与 TileLang 社区紧密合作，以促进更敏捷、高效和稳定的核开发工作流程。

使用宿主代码生成器降低调用开销。随着加速器性能的不断提升，CPU 端的编排开销日益凸显。对于小型、高度优化的内核而言，这种固定的宿主开销很容易限制利用率和吞吐量。这种开销的一个常见来源是，为了灵活性，宿主端逻辑（如运行时合约检查）通常使用 Python 编写，从而产生固定的每次调用成本。

我们通过 *Host Codegen* 来减轻这一开销，它将大部分主机端逻辑迁移到生成的主机代码中。具体而言，我们首先在 IR（中间表示）级别共同生成

设备内核和一个轻量级的主机启动器，嵌入从语言前端解析得到的必要元数据，如数据类型、秩/形状约束以及步长/布局假设。然后，将启动器降级为基于 TVM-FFI (Chen 等人, 2018) 框架构建的主机源代码，该框架的紧凑调用约定和零拷贝张量互操作共同将主机端开销降至最低。在运行时，这些生成的主机代码执行验证和参数封送处理，将所有每次调用的检查从 Python 执行路径中移除。我们的测量结果显示，CPU 端的验证开销从每次调用数十或数百微秒降低到不到一微秒。

SMT 求解器辅助的形式化整数分析。 TileLang 内核涉及复杂的张量索引运算，需要强大的形式化整数分析。在布局推理、内存危险检测和边界分析等编译过程中，编译器必须验证整数表达式是否满足特定属性，以启用相应的优化。因此，更强大的形式化分析能力可以解锁更多高级且复杂的优化机会。

为此，我们将 Z3 SMT 求解器 (De Moura 和 Bjørner, 2008) 集成到 TileLang 的代数系统中，为张量程序中的大多数整数表达式提供形式化分析能力。我们通过将 TileLang 的整数表达式转换为 Z3 的无量词非线性整数算术 (QF_NIA)，在计算开销和形式化表达能力之间取得了平衡。基于整数线性规划 (ILP) 求解器，QF_NIA 能够无缝解析内核中常见的标准线性整数表达式。此外，其固有的非线性推理能力有效地解决了诸如可变张量形状上的向量化等高级挑战。在合理的资源限制下，Z3 提升了整体优化性能，同时将编译时间开销限制在仅几秒钟。这一影响在多个阶段都十分显著，包括向量化、屏障插入和代码简化。

数值精度与位级可复现性。 在生产环境中，数值正确性和可复现性与原始吞吐量同样重要。因此，我们默认优先考虑准确性：在编译器层面禁用快速数学优化，并且仅将影响精度的近似值作为显式的、可选择加入的前端运算符提供（例如，`T.__exp`、`T.__log` 和 `T.__sin`）。相反，当需要严格的 IEEE-754 语义时，TileLang 提供符合 IEEE 标准的内部函数，并带有显式的舍入模式（例如，`T.ieee_fsqrt`、`T.ieee_fdiv` 和 `T.ieee_add`），使开发人员

能够精确指定数值行为。

我们还致力于实现位级可复现性，以便针对手写的 CUDA 基线来验证内核。我们将 TileLang 的代数简化和降级规则与主流 CUDA 工具链（如 NVCC）对齐，以避免引入意外的位级差异的转换。布局注释（如 `T.annotate_layout`）进一步允许用户确定依赖于布局的降级决策，使评估和累积顺序与参考 CUDA 实现保持一致，从而在需要时实现位级相同的输出。

我们的评估表明，这些以准确性和可重复性为导向的设计选择并不会牺牲性能：在保守的默认设置下，TileLang 内核仍然具有竞争力，同时提供了旋钮以选择性地放宽数值约束，从而获得更高的速度。

3.3. 高性能批量不变且确定性的内核库

为了实现高效的训练和推理，我们开发了一套全面的高性能计算内核。除了基本功能和最大化硬件利用率外，另一个关键设计目标是确保训练的可重复性以及预训练、后训练和推理流程之间的位对齐。因此，我们以最小的性能开销实现了端到端、位级批量不变且确定性的内核。这些内核有助于调试、稳定性分析和保持一致的后训练行为。

批量不变性。批量不变性确保任何给定标记的输出在位级上保持一致，无论其在批量中的位置如何。为实现批量不变性，主要挑战如下：

- **注意。**为了实现批量不变性，我们不能使用分割键值（split-KV）方法（Dao 等人，2023），该方法将单个序列的注意力计算分布在多个流多处理器（SM）上，以平衡 SM 的负载。然而，放弃这种技术将导致严重的波量化问题³，这会对 GPU 利用率产生不利影响。为了解决这一问题，我们开发了一种双核策略来实现批量不变解码。第一个核在单个 SM 内计算整个序列的注意力输出，确保满载波的高吞吐量。第二个核使用多个 SM 处理单个序列，以最小化最终部分填充波的延迟，从而减轻波量化问题。为了保持这两个核在位级上的一致性，我们精心设计了第二个核的计算路

³<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant>

径，以确保其累加顺序与第一个核相同。此外，第二个核利用线程块簇内的分布式共享内存⁴，实现 SM 间的高速数据交换。这种双核方法有效地将批量不变解码的开销限制在可忽略不计的水平。

- **矩阵乘法。**传统的 cuBLAS 库 (NVIDIA Corporation, 2024) 无法实现批量不变性。因此，我们将其端到端地替换为 DeepGEMM (Zhao 等人, 2025)。此外，对于非常小的批量大小，传统实现通常采用 split-k (Osama 等人, 2023) 技术来提高性能。不幸的是，split-k 技术无法保证批量不变性，而这是 DeepSeek-V4 中的一个关键特性。

因此，在大多数场景下，我们放弃了分块 k，但这可能会导致性能下降。为了解决这一问题，我们引入了一系列优化措施，使我们的矩阵乘法实现能够在大多数主要场景下达到甚至超越标准分块 k 的性能。

确定性。确定性训练对于调试硬件或软件问题非常有益。此外，当训练过程中出现异常情况（如损失激增）时，确定性使研究人员能够更容易地查明数值原因，并进一步完善模型设计。训练中的非确定性通常源于非确定性累积顺序，这通常是由于使用了原子加法指令。该问题主要发生在反向传播过程中，特别是在以下部分：

- **注意力反向传播。**在稀疏注意力反向传播的传统实现中，我们使用原子加法来累积键值 (KV) 标记的梯度。由于浮点加法的非结合性，这引入了非确定性。为了解决这个问题，我们为每个源映射 (SM) 分配单独的累积缓冲区，然后在所有缓冲区上进行全局确定性求和。
- **混合专家 (Mixture of Experts, MoE) 反向传播。**当来自不同秩的多个子模型 (Sub-Models, SMs) 同时向接收秩上的同一缓冲区写入数据时，协商写入位置也会引入非确定性。为了解决这一问题，我们在每个单独的秩内设计了一种标记顺序预处理机制，并结合了跨多个秩的缓冲区隔离。这一策略确保了专家并行发送结果和 MoE 反向传播中累积顺序的确定性。

⁴<https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#distributed-shared-memory>

- **矩阵乘法 in mHC .** mHC 涉及一个输出维度仅为 24 的矩阵乘法。对于非常小的批量大小，我们不得不使用 split-k (Osama 等人, 2023) 算法，其简单实现会导致非确定性。为了克服这一点，我们分别输出每个拆分部分，并在后续内核中进行确定性归约，从而既保持性能又保持确定性。

3.4. FP4 量化感知训练

为了在部署时实现推理加速和内存节省，我们在后训练阶段引入了量化感知训练 (QAT) (Jacob 等人, 2018)，使模型能够适应量化带来的精度降低。我们将 FP4 (MXFP4) 量化 (Rouhani 等人, 2023) 应用于两个组件：(1) MoE 专家权重，这是 GPU 内存占用的主要来源 (OpenAI, 2025)，以及 (2) CSA 索引器中的查询-键 (QK) 路径，其中 QK 激活完全在 FP4 中缓存、加载和相乘，从而加速长上下文场景中的注意力得分计算。此外，在此 QAT 过程中，我们将索引得分 $I_{:, :}$ 从 FP32 进一步量化为 BF16。此优化使 top-k 选择器的速度提高了 $2\times$ 倍，同时保持了 KV 条目 99.7% 的召回率。

对于混合专家 (MoE) 的权重，遵循量化感知训练 (QAT) 的常见做法，优化器维护的 FP32 主权重首先被量化为 FP4，然后反量化回 FP8 以进行计算。值得注意的是，我们的 FP4 到 FP8 的反量化是无损的。这是因为与 FP4 (E2M1) 相比，FP8 (E4M3) 多出了 2 个指数位，提供了更大的动态范围。因此，只要每个 FP8 量化块 (128×128 个块) 内 FP4 子块 (1×128 个块) 的最大和最小比例因子之间的比率不超过某个阈值，那么细粒度的比例信息就可以被 FP8 的扩展动态范围完全吸收。我们通过实验验证了当前的权重满足这一条件。这使得整个 QAT 流程能够在不进行任何修改的情况下，完全复用现有的 FP8 训练框架。在反向传播中，相对于前向传播中的相同 FP8 权重计算梯度，并直接将其传播回 FP32 主权重，相当于通过量化操作应用直通估计器 (STE)。这也避免了重新量化转置权重的需要。

在强化学习 (RL) 训练的推理和滚动阶段 (这些阶段不涉及反向传播)，我们直接使用真实的 FP4 量化权重，而不是模拟量化。这确保了采样过程

中的模型行为与在线部署完全一致，同时减少了内核内存加载以实现实际加速，并显著降低了内存消耗。我们同样在 CSA 的索引器中处理 QK 路径。

3.5. 训练框架

我们的训练框架建立在为 DeepSeek-V3 (DeepSeek-AI, 2024) 开发的可扩展且高效的基础设施之上。在训练 DeepSeek-V4 时，我们继承了这一坚实基础，同时引入了几项关键创新以适应其新颖的架构组件——特别是 Muon 优化器、*mHC* 和混合注意力机制——同时保持了高训练效率和稳定性。

3.5.1. *Muon* 的高效实现

Muon 优化器需要完整的梯度矩阵来计算参数更新，当与零冗余优化器 (ZeRO) (Rajbhandari 等人, 2020) 结合使用时，这带来了挑战。传统的 ZeRO 是为像 AdamW 这样的元素级优化器设计的，其中单个参数矩阵可以被划分并在多个秩上进行更新。为了解决这一冲突，我们为 Muon 设计了一种 ZeRO 桶分配的混合策略。

对于密集参数，我们限制 ZeRO 并行性的最大规模，并采用背包算法为这些秩分配参数矩阵，确保每个秩管理大致均衡的负载。每个秩上的桶会被填充，以匹配跨秩的最大桶的大小，从而促进高效的归约-散播操作。在我们的设置中，每个秩管理不超过五个参数矩阵，这种填充通常只会带来不到 10% 的内存开销。当数据并行性的总体规模超过 ZeRO 的限制时，我们会在额外的数据并行组中冗余地计算 Muon 更新，以计算换取减少的总桶内存。

对于混合专家 (Mixture of Experts, MoE) 参数，我们独立优化每个专家。首先，我们将所有专家在所有层中的 SwiGLU (Shazeer, 2020) 中的所有下投影矩阵展平，然后是展平的上投影矩阵和门矩阵。然后，我们对展平的向量进行填充，以确保我们可以在所有秩上均匀分布该向量，而不会拆分任何逻辑上独立的矩阵。鉴于专家数量众多，我们对混合专家参数不施加 ZeRO 并行性的限制，且填充开销也可忽略不计。

此外，在每个秩上，相同形状连续参数将自动合并，从而能够批量执行 Newton-Schulz 迭代以更好地利用硬件资源。此外，我们观察到，在 Muon 中，使用 BF16 矩阵乘法计算时，Newton-Schulz 迭代保持稳定。基于此，我们进一步以随机舍入的方式，将数据并行秩之间同步的混合专家（Mixture of Experts, MoE）梯度量化到 BF16 精度，使通信量减半。为了避免低精度加法器引入的累积误差，我们采用两阶段方法替代了传统的基于树或环的 reduce-scatter 集合通信。首先，通过全对全操作在秩之间交换局部梯度，然后每个秩在 FP32 中执行局部求和。这种设计保持了数值的鲁棒性。

3.5.2. *mHC* 的成本效益和内存效率实现

与传统的残差连接相比，*mHC* 的引入增加了激活内存消耗和流水线阶段之间的通信量。为了降低这些成本，我们实施了多种优化策略。

首先，我们精心设计和实现了用于训练和推理的融合卷积核（*mHC*）。其次，我们引入了一种重新计算策略，该策略选择性地检查中间张量。具体而言，我们重新计算层间的大多数隐藏状态和所有归一化层输入，同时避免重新计算计算密集型操作。这实现了内存节省和计算开销之间的平衡。第三，我们调整了 DualPipe 1F1B 重叠方案，以适应增加的流水线通信，并支持 *mHC* 中某些操作的并发执行。

总体而言，这些优化将 *mHC* 的墙时间开销限制在重叠 1F1B 流水线阶段的仅 6.7%。有关工程优化的更多详细信息，请参阅专门的 *mHC* 论文（Xie 等人，2026）。

3.5.3. 长上下文中的上下文并行性注意

传统的上下文并行性（CP）对序列维度进行划分，每个秩保持连续的 S 个标记。这给我们的压缩注意力机制（即 CSA 和 HCA）带来了两个挑战。一方面，训练样本是从多个序列中打包得到的，每个序列独立压缩 M 倍（或 M' 倍），任何少于 M 的尾部标记都会被丢弃。因此，压缩后的 KV（键值）

长度通常小于 S_M ，并且在不同的秩之间有所不同。另一方面，压缩需要 M 个连续的 KV 条目，这些条目可能跨越两个相邻 CP 秩之间的边界。

为了应对这些挑战，我们设计了一种两阶段通信方法。在第一阶段，每个秩 I 将其最后 M 个未压缩的键值 (KV) 条目发送给秩 $i+1$ 。然后，秩 $I+1$ 将这些接收到的条目中的一部分与其本地 S 个未压缩的 KV 条目一起压缩，生成固定长度为 $\frac{s}{m}+1$ 的压缩条目，其中包含一些填充条目。在第二阶段，所有 CP 秩之间的全收集操作会收集本地压缩的 KV 条目。然后，一个融合的选择与填充算子将它们重新组织成总长度为 $\text{cp_size} \cdot \frac{s}{m}$ 的完整压缩 KV 条目集。任何填充条目都放在尾部。对于 HCA 和 CSA 中的索引器，每个查询令牌的压缩 KV 条目的可见范围可以通过规则预先计算。对于 CSA 中的稀疏注意力机制，top- K 选择器会明确指定每个查询的可见压缩 KV 条目的索引。

3.5.4. 扩展自动微分以实现灵活的激活检查点

传统的激活检查点实现以整个模块为粒度，决定在反向传播过程中是保留还是重新计算其输出激活。这种粗粒度通常会导致重新计算成本和激活内存占用之间的次优权衡。另一种方法是手动实现整个层的前向和后向逻辑，显式管理张量检查点状态。虽然这种方法能够实现细粒度控制，但它失去了自动微分框架的便利性，大大增加了开发的复杂性。

为了在不牺牲编程效率的情况下实现细粒度控制，我们实现了一种具有自动微分支持的张量级激活检查点机制。借助此机制，开发人员只需实现前向传递，并选择性地为单个张量添加注释以进行自动检查点和重新计算。我们的框架利用 TorchFX (Reed 等人, 2022) 来追踪完整的计算图。对于每个带注释的张量，它会执行向后遍历以识别其重新计算所需的最小子图。我们将这些最小子图定义为重新计算图，并将其插入到相应梯度计算之前的向后逻辑中。

与手动实现相比，本设计在训练过程中不引入任何额外开销。在此框架

中，重计算是通过直接释放已标注张量的 GPU 内存并重用重计算张量的存储指针来实现的，无需进行任何 GPU 内存复制。此外，由于图跟踪会具体执行模型，我们可以跟踪每个张量的底层存储指针，从而实现对共享存储的张量（例如，重塑操作的输入和输出）的重计算进行自动去重。这使开发人员在标注重计算时无需考虑低级内存的细节。

3.6. 推理框架

我们的推理框架在很大程度上继承了 DeepSeek-V3 的框架，但在 KV Cache 管理方面存在一些差异。

3.6.1. KV 缓存结构与管理

为了高效管理 DeepSeek-V4 中混合注意力机制产生的异构 KV 缓存，我们设计了一种定制的 KV 缓存布局。该布局如图 6 所示，我们将详细阐述如下。

DeepSeek-V4 中的异构键值对 (KV) 条目。 DeepSeekV4 系列中的混合注意力机制引入了多种类型的 KV 条目，这些条目具有不同的键值 (KV) 缓存大小和更新规则。用于稀疏选择的闪电索引器在 KV 缓存中引入了额外的维度，这些维度的嵌入大小与主注意力中的嵌入大小不同。压缩策略分析 (CSA) 和混合压缩分析 (HCA) 中采用的压缩技术分别将序列长度缩短了 $1/M$ 倍和 $1/M'$ 倍，从而减小了整体 KV 缓存的大小。因此，不同层的 KV 缓存大小各不相同。此外，滑动窗口注意力 (SWA) 层也使用不同的 KV 缓存大小，以及独立的缓存命中和淘汰策略。在压缩分支中，每 M 个标记生成一个 KV 条目。当剩余标记数量不足以进行压缩时，所有待处理的标记及其相关的隐藏状态必须保留在缓冲区中，直到可以执行压缩操作。这些缓冲区的标记表示由位置上下文确定的序列状态，也在 KV 缓存框架内进行管理。

混合注意力 KV 缓存管理中的挑战。混合注意力机制违背了 PagedAttention

及其变体背后的基本假设。尽管最近的混合 KV 缓存管理算法（如 Jenga (Zhang 等人, 2025a)、Hymba (Dong 等人, 2025)）针对通用混合注意力模型或特定结构，但在 PagedAttention 框架下，有两个主要障碍阻碍了 KV 缓存跨层整合：

- 多样化的缓存策略，如滑动窗口注意力（Sliding Window Attention）中所使用的策略。
- 高性能注意力核所施加的约束，包括对齐要求。

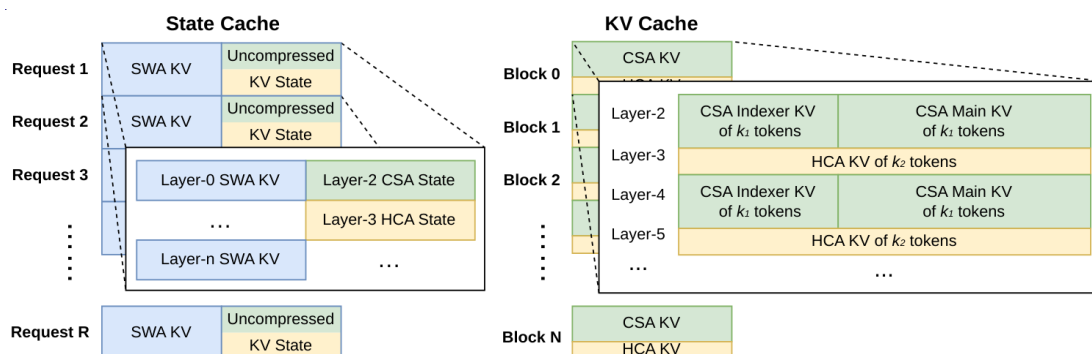


图 6 | DeepSeek-V4 的 KV 缓存布局示意图。KV 缓存主要由两部分组成：用于 CSA/HCA 的传统 KV 缓存，以及用于 CSA/HCA 中 SWA 和未压缩就绪标记的状态缓存。在状态缓存中，每个请求被分配一个固定大小的缓存块。在该块内，SWA 段存储与最新 n_{win} 个标记对应的 KV 条目，而 CSA/HCA 段存储尚未压缩就绪的未压缩尾部状态。在传统 KV 缓存中，我们为每个请求分配多个块。每个缓存块覆盖 $\text{lcm}(M, M')$ 个原始标记，生成 $k_1 = \frac{\text{lcm}(m, m')}{m}$ 个 CSA 压缩标记和 $k_2 = \frac{\text{lcm}(m, m')}{m'}$ 个 HCA 压缩标记。

为了有效管理 DeepSeek-V4 的键值（KV）缓存，我们设计了相应的策略来克服这两个挑战。

状态缓存用于 SWA 和未压缩尾部标记。为解决第一个障碍，我们采用了一种替代的缓存管理机制。由于 SWA 旨在在有限的键值（KV）缓存大小下提升性能，因此将其与压缩分支中的未压缩尾部标记一起视为状态空间模型是合理的。相应的 KV 缓存因此可以被视为仅依赖于当前位置的序列特定状态。据此，我们预分配了一个固定且有限大小的状态缓存池，并动态地将其分配给每个序列。

稀疏注意力核协同设计。针对第二个障碍，传统的高性能注意力核通常假设

每个块有固定数量的 B 个标记以优化性能，这在 CSA 中对应于 $B \cdot M$ 个原始标记，在 HCA 中对应于 $B \cdot M'$ 个原始标记。通过采用高性能稀疏注意力核，不同层可以在每个块中容纳可变数量的标记，而不会降低性能。实现这一点需要协同设计 KV 缓存布局 and 稀疏注意力核。例如，通过填充块以与缓存行对齐可以提高性能。因此，对于压缩比为 M 的 CSA 和压缩比为 M' 的 HCA，每个块的原始标记数量可以是这两个压缩比的最小公倍数 $\text{lcm}(M, M')$ 的任意倍数。

3.6.2. 磁盘 KV 缓存存储

在提供 DeepSeek-V4 服务时，我们利用磁盘 KV 缓存存储机制来消除对共享前缀请求的重复预填充。对于 CSA/HCA 中的压缩 KV 条目和滑动窗口注意力 (SWA) 中的未压缩 KV 条目，我们设计了独立的存储管理解决方案。

对于 CSA 和 HCA，我们只需将所有压缩后的 KV 条目存储到磁盘中。当请求命中已存储的前缀时，我们会读取并重用与该前缀对应的压缩 KV 条目，直到最后一个完整的压缩块。特别地，对于尾部不完整块中的前缀标记，我们仍需要重新计算它们以恢复未压缩的 KV 条目，因为在 CSA 和 HCA 中未压缩的 KV 条目并未存储。

对于 SWA KV 条目，由于它们未被压缩且存在于每一层中，因此其体积大约是压缩后的 CSA 和 HCA KV 条目的 8 倍。为了高效处理这些庞大的 SWA KV 条目，我们提出并实施了三种不同的策略来管理磁盘上的 SWA KV 条目，每种策略都在存储开销和计算冗余之间提供了不同的权衡：

- **全 SWA 缓存。**此策略存储所有令牌的完整 SWA 键值 (KV) 条目，确保计算零冗余。在此策略下，只需读取该前缀内最后 n_{wir} 个令牌的磁盘缓存，即可重建命中前缀的 SWA KV 条目。尽管实现了计算零冗余，但此策略对于现代基于固态硬盘 (SSD) 的存储系统而言效率低下——对于每个命中请求，仅会访问所存储的 SWA KV 缓存中的一小部分，这导致了一种不平衡的写密集型访问模式。

- **周期性检查点**。该策略在每 P 个令牌内检查点最后 N 个获胜令牌的 SWA KV 条目，其中 P 是一个可调参数。对于命中前缀，我们加载最近检查点的状态，然后重新计算剩余的尾部令牌。通过调整 P ，该策略能够在存储和计算之间实现按需权衡。
- **零 SWA 缓存**。此策略不存储任何 SWA 键值 (KV) 条目。对于命中前缀，我们需要执行更多重新计算来恢复 SWA KV 条目。具体而言，在每个注意力层中，每个标记的 SWA KV 条目仅依赖于上一层最近 N 个窗口标记的 SWA KV 条目。因此，对于 L 层模型，利用缓存的 CSA 和 HCA KV 条目，重新计算最后 $n_{\text{win}} \cdot L$ 个标记就足以恢复最后 n_{win} 个 SWA KV 条目。

根据具体的部署场景，我们选择最合适的策略来实现存储与计算之间的理想平衡。

4. 预训练

4.1. 数据构建

在 DeepSeek-V3 的预训练数据基础上，我们致力于构建一个更加多样、质量更高且有效上下文更长的训练语料库。我们持续优化数据构建流程。对于网络来源的数据，我们实施了过滤策略，以去除批量自动生成和模板化的内容，从而降低模型崩溃的风险 (Zhu 等人, 2024)。数学和编程语料库仍然是我们的训练数据核心组成部分，我们通过在训练中期加入代理数据，进一步增强了 DeepSeek-V4 系列的编码能力。对于多语言数据，我们为 DeepSeek-V4 构建了一个更大的语料库，以提升其跨不同文化捕捉长尾知识的能力。对于 DeepSeek-V4，我们特别强调长文档数据的整理，优先选择科学论文、技术报告和其他反映独特学术价值的材料。综合上述因素，我们的预训练语料库包含超过 32T 的标记，涵盖数学内容、代码、网页、长文档和其他高质量类别。

对于预训练数据，我们主要遵循 DeepSeek V3 的预处理策略。在分词方

面，我们在 DeepSeek-V3 分词器的基础上，引入了一些特殊标记用于构建上下文，同时仍将词汇表大小保持在 128K。我们还继承了 DeepSeek-V3 中的分词策略 (DeepSeek-AI, 2024) 和中间填充 (FIM) (DeepSeek-AI, 2024)。受 Ding 等人 (2024) 的启发，我们将来自不同来源的文档打包成适当的序列，以最大限度地减少样本截断。与 DeepSeek-V3 不同，我们在预训练过程中采用了样本级注意力掩码。

4.2. 预训练设置

4.2.1. 模型设置

DeepSeek-V4-Flash。我们将 Transformer 层的数量设置为 43，隐藏维度 D 设置为 4096。对于前两层，我们使用纯滑动窗口注意力。对于后续层，以交错方式使用 CSA 和 HCA。对于 CSA，我们将压缩率 M 设置为 4，索引查询头数量 n_h 设置为 64，索引头维度 C^I 设置为 128，以及为稀疏注意力选择的 KV 条目数量（即注意力 top-k）设置为 512。对于 HCA，我们将压缩率 M' 设置为 128。对于 CSA 和 HCA，我们将查询头数量 n_h 设置为 64，头维度 C 设置为 512，以及查询压缩维度 D_C 设置为 1024。输出投影组 G 的数量设置为 8，每个中间注意力输出 d_g 的维度设置为 1024。对于滑动窗口注意力的额外分支，窗口大小 n_{win} 设置为 128。我们在所有 Transformer 块中都使用了 MoE 层，但对于前 3 个 MoE 层使用了哈希路由策略。每个 MoE 层由 1 个共享专家和 256 个路由专家组成，其中每个专家的中间隐藏维度为 2048。在路由专家中，每个标记将激活 6 个专家。多标记预测深度设置为 1。至于 mHC ，扩展因子 n_{hc} 设置为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设置为 20。在此配置下，DeepSeek-V4-Flash 总共有 284B 参数，其中每个标记激活 13B 参数。

DeepSeek-V4-Pro。我们将 Transformer 层数设置为 61 层，隐藏维度 D 设置为 7168。对于前两层，我们使用 HCA（混合注意力）。对于后续层，CSA（稀疏注意力）和 HCA 以交错方式使用。对于 CSA，我们将压缩率

M 设置为 4，索引查询头数量 N^I 设置为 64，索引头维度 d^I 设置为 128，以及为稀疏注意力选择的 KV 条目数量（即注意力 top-k）设置为 1024。对于 HCA，我们将压缩率 M' 设置为 128。对于 CSA 和 HCA，我们将查询头数量 N 设置为 128，头维度 C 设置为 512，查询压缩维度 D_C 设置为 1536。输出投影组 G 的数量设置为 16，每个中间注意力输出 d_g 的维度设置为 1024。对于滑动窗口注意力的额外分支，窗口大小 N_{win} 设置为 128。我们在所有 Transformer 块中采用 MoE（混合专家）层，但前 3 个 MoE 层使用哈希路由策略。每个 MoE 层由 1 个共享专家和 384 个路由专家组成，其中每个专家的中间隐藏维度为 3072。在路由专家中，每个标记将激活 6 个专家。多标记预测深度设置为 1。至于 mHC （多头注意力），扩展因子 n_{hc} 设置为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设置为 20。在此配置下，DeepSeek-V4-Flash 包含 1.6T 总参数，其中每个标记激活 49B 参数。

4.2.2. 训练设置

DeepSeek-V4-Flash。我们为大多数参数使用 Muon 优化器（Jordan 等人，2024；Liu 等人，2025），但为嵌入模块、预测头模块和所有 RMSNorm 模块的权重使用 AdamW 优化器（Loshchilov 和 Hutter，2017）。对于 AdamW，我们将超参数设置为 $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-20}$ ，且 `weight_decay` 设置为 0.1。对于 Muon，我们将动量设置为 0.95，权重衰减设置为 0.1，并将每次更新矩阵的 RMS 重新缩放为 0.18，以重新利用 AdamW 的学习率。我们在 32T 个标记上训练 DeepSeek-V4-Flash，并且与 DeepSeek-V3 一样，我们还采用了批量大小调度策略，该策略将批量大小（以标记为单位）从小批量增加到 75.5M，然后在大部分训练过程中保持 75.5M 不变。学习率在前 2000 步中线性升温，在大部分训练过程中保持在 2.7×10^{-4} 。在训练接近尾声时，我们最终按照余弦调度将学习率衰减到 2.7×10^{-5} 。训练开始时的序列长度为 4K，我们逐渐将训练序列长度扩展到 16K、64K 和 1M。至于稀疏注意力设置，我们首先在前 1T 个标记中使用稠密注意力对模型进行预热，然后在序列长度为 64K 时引入稀疏注意力，并在剩余训练过程中保持稀疏注意力。在引入注意力稀疏性时，我们首先设置一个短阶段来预热 CSA 中的闪电索

引器，然后在大部分训练过程中使用稀疏注意力训练模型。对于无辅助损失的负载均衡，我们将偏差更新速度设置为 0.001。对于平衡损失，我们将损失权重设置为 0.0001，以避免单个序列中的极端不平衡。在大部分训练过程中，MTP 损失权重设置为 0.3，而在学习率开始衰减时设置为 0.1。

DeepSeek-V4-Pro。除特定超参数值外，DeepSeek-V4-Pro 的训练设置与 DeepSeek-V4-Flash 基本一致。我们对大多数参数采用 Muon 优化器，但对嵌入模块、预测头模块以及所有 RMSNorm 模块的权重采用 AdamW 优化器。AdamW 和 Muon 的超参数与 DeepSeek-V4-Flash 的相同。我们在 33T 个标记上训练 DeepSeek-V4-Pro，并采用批量大小调度策略，最大批量大小为 94.4M 个标记。学习率调度策略与 DeepSeek-V4-Flash 基本相同，但峰值学习率设置为 2.0×10^{-4} ，最终学习率设置为 2.0×10^{-5} 。训练也从序列长度为 4K 开始，逐渐延长至 16K、64K 和 1M。与 DeepSeek-V4-Flash 相比，DeepSeek-V4-Pro 从更长的密集注意力阶段开始，引入稀疏注意力的策略与 DeepSeek-V4-Flash 相同，采用两阶段训练方法。对于无辅助损失的负载均衡，我们将偏置更新速度设置为 0.001。对于平衡损失，我们将损失权重设置为 0.0001，以避免单个序列内极端不平衡。在大多数训练过程中，MTP 损失权重设置为 0.3，在学习率开始衰减时设置为 0.1。

4.2.3. 缓解训练不稳定性

训练万亿参数的 MoE 模型面临着巨大的稳定性挑战，DeepSeekV4 系列也不例外。在训练过程中，我们遇到了显著的不稳定性挑战。虽然简单的回滚可以暂时恢复训练状态，但它们作为长期解决方案并不足够，因为它们并不能防止损失峰值的再次出现。从实证角度来看，我们发现峰值的出现始终与 MoE 层中的异常值相关，而路由机制本身似乎加剧了这些异常值的出现。因此，我们试图从两个方面来解决这个问题：打破由路由引起的恶性循环，以及直接抑制异常值。幸运的是，我们发现了两种实用的技术，它们能够有效地保持训练的稳定性。尽管对其潜在机制的全面理论理解目前仍是一个未

解决的问题，但我们公开分享这些技术，以促进社区的进一步探索。

预期路由。我们发现，将骨干网络和路由网络的同步更新解耦可显著提高训练稳定性。因此，在第 T 步，我们使用当前网络参数 θ_T 进行特征计算，但路由索引是使用历史网络参数 $\theta_{t-\Delta}$ 计算并应用的。在实践中，为了避免两次加载模型参数的开销，我们在第 T 步 $-\Delta T$ 提前获取第 T 步的数据。我们“预期地”计算并缓存路由索引，以备在第 T 步时使用，因此我们将这种方法命名为预期路由。此外，我们在基础设施层面进行了大量优化。首先，鉴于预计算路由索引只需要对数据进行一次前向传播，我们精心编排了流水线执行以及计算与专家并行（Expert Parallelism, EP）通信的重叠，成功地将预期路由带来的额外时钟时间开销限制在约 20%。其次，我们引入了一种自动检测机制，该机制仅在发生损失激增时触发短暂回退并激活预期路由；在此模式下运行一段时间后，系统将恢复标准训练。最终，这种动态应用使我们能够在几乎不增加额外训练开销的情况下避免损失激增，同时不损害模型性能。

SwiGLU 夹紧。在之前的文献中（Bello 等人，2017 年；Riviere 等人，2024 年），夹紧被明确用于约束数值范围，从而增强训练稳定性。在我们实际的训练运行中，我们实证发现，应用 SwiGLU 夹紧（OpenAI，2025 年）可有效消除异常值，并在不损害性能的情况下，显著有助于稳定训练过程。在 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的训练过程中，我们将 SwiGLU 的线性分量夹紧在 $[-10, 10]$ 的范围内，同时将门控分量的上限限制为 10。

4.3. 评估

4.3.1. 评估基准

对于基础模型的评估，我们考虑了涵盖四个关键维度的基准：世界知识、语言理解和推理、编码和数学，以及长上下文处理。

世界知识基准测试包括 AGIEval（Zhong 等人，2023）、C-Eval（Huang 等人，2023）、CMMLU（Li 等人，2023）、MMLU（Hendrycks 等人，2020）、

MMLU-Redux (Gema 等人, 2024)、MMLU-Pro (Wang 等人, 2024b)、MMMLU (OpenAI, 2024a)、MultiLoKo (Hupkes 和 Bogoychev, 2025)、Simple-QA 验证版 (Haas 等人, 2025)、SuperGPQA (Du 等人, 2025)、FACTS Parametric (Cheng 等人, 2025) 以及 TriviaQA (Joshi 等人, 2017)。

语言理解和推理基准测试包括 BigBench Hard (BBH) (Suzgun 等人, 2022 年)、DROP (Dua 等人, 2019 年)、HellaSwag (Zellers 等人, 2019 年)、CLUEWSC (Xu 等人, 2020 年) 和 WinoGrande (Sakaguchi 等人, 2019 年)。

编码和数学基准测试包括 BigCodeBench (Zhuo 等人, 2025 年)、HumanEval (Chen 等人, 2021 年)、GSM8K (Cobbe 等人, 2021 年)、MATH (Hendrycks 等人, 2021 年)、MGSM (Shi 等人, 2023 年) 和 CMath (Wei 等人, 2023 年)。

长文本基准测试包括 LongBench-V2 (Bai 等人, 2025b)。

表 1 | DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base 和 DeepSeek-V4Pro-Base 之间的比较。所有模型均在我们的内部框架中进行评估，并采用相同的评估设置。分数差距不超过 0.3 的视为同一水平。每行中得分最高的用**粗体**表示，第二高的用下划线表示。

Benchmark (Metric)		# Shots	DeepSeek-V3.2 Base	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base
Architecture		-	MoE	MoE	MoE
# Activated Params		-	37B	13B	49B
# Total Params		-	671B	284B	1.6T
World Knowl.	AGIEval (EM)	0-shot	80.1	<u>82.6</u>	83.1
	MMLU (EM)	5-shot	87.8	<u>88.7</u>	90.1
	MMLU-Redux (EM)	5-shot	87.5	<u>89.4</u>	90.8
	MMLU-Pro (EM)	5-shot	65.5	<u>68.3</u>	73.5
	MMMLU (EM)	5-shot	87.9	<u>88.8</u>	90.3
	C-Eval (EM)	5-shot	90.4	<u>92.1</u>	93.1
	CMMLU (EM)	5-shot	88.9	<u>90.4</u>	90.8
	MultiLoKo (EM)	5-shot	38.7	<u>42.2</u>	51.1
	Simple-QA verified (EM)	25-shot	28.3	<u>30.1</u>	55.2
	SuperGPQA (EM)	5-shot	45.0	<u>46.5</u>	53.9
	FACTS Parametric (EM)	25-shot	27.1	<u>33.9</u>	62.6
	TriviaQA (EM)	5-shot	<u>83.3</u>	82.8	85.6
Lang. & Reas.	BBH (EM)	3-shot	87.6	<u>86.9</u>	87.5
	DROP (F1)	1-shot	<u>88.2</u>	88.6	88.7
	HellaSwag (EM)	0-shot	<u>86.4</u>	85.7	88.0
	WinoGrande (EM)	0-shot	78.9	<u>79.5</u>	81.5
	CLUEWSC (EM)	5-shot	<u>83.5</u>	82.2	85.2
Code & Math	BigCodeBench (Pass@1)	3-shot	63.9	56.8	<u>59.2</u>
	HumanEval (Pass@1)	0-shot	62.8	<u>69.5</u>	76.8
	GSM8K (EM)	8-shot	<u>91.1</u>	90.8	92.6
	MATH (EM)	4-shot	<u>60.5</u>	57.4	64.5
	MGSM (EM)	8-shot	81.3	85.7	<u>84.4</u>
	CMath (EM)	3-shot	<u>92.6</u>	93.6	90.9
Long Context	LongBench-V2 (EM)	1-shot	40.2	<u>44.7</u>	51.5

4.3.2. 评估结果

在表 1 中，我们提供了 DeepSeek-V3.2、DeepSeekV4-Flash 和 DeepSeek-V4-Pro 基础模型的详细比较，所有模型均在统一的内部框架下进行评估，且设置严格一致。

将 DeepSeek-V4-Flash-Base 与 DeepSeek-V3.2-Base 进行比较，展现了令人信服的效率优势。尽管 DeepSeek-V4-Flash-Base 在激活参数和总参数数量上大幅减少，但在一系列广泛的基准测试中，其表现优于 DeepSeek-V3.2-Base。这一优势在世界知识任务和具有挑战性的长上下文场景中尤为明显。这些结果强调，即使在参数预算更为紧凑的情况下，DeepSeek-V4-Flash-Base 中的架构改进、数据质量的提升以及训练优化也能带来更优的性能，在大多数评估中有效超越了规模更大的 DeepSeek-V3.2-Base。

此外，DeepSeek-V4-Pro-Base 在能力上实现了进一步的决定性飞跃，几乎全面超越了 DeepSeek-V3.2-Base 和 DeepSeek-V4-FlashBase。DeepSeek-V4-Pro-Base 在几乎所有类别上都有所改进，在最苛刻的基准测试中，其性能达到了 DeepSeek 基础模型的新高度。在知识密集型评估中，它取得了显著成果，同时也大幅提升了长上下文理解能力。在大多数推理和代码基准测试中，DeepSeek-V4-Pro-Base 的表现也超过了之前的两个模型。这一全面的提升证实了 DeepSeek-V4-Pro-Base 是 DeepSeek 系列中最强大的基础模型，在知识、推理、编码和长上下文能力方面均超越了其前身。

5. 后训练

5.1. 后训练流程

在预训练之后，我们进行了后训练阶段，以生成 DeepSeekV4 系列的最终模型。尽管训练流程在很大程度上反映了 DeepSeek-V3.2 的流程，但进行了一项关键的方法论替换：混合强化学习（RL）阶段完全被策略内蒸馏（OPD）所取代。

5.1.1. 专家训练

领域专家的开发是通过调整 DeepSeek-V3.2 训练流程来实现的。具体而言，每个模型都通过初始微调阶段和后续的强化学习（RL）阶段进行顺序优化，强化学习阶段由领域特定的提示和奖励信号引导。对于强化学习阶段，我们实现了组相对策略优化（GRPO）算法，并保持了与先前研究（DeepSeek-AI, 2025; DeepSeek-AI, 2025）高度一致的超参数。

推理能力。人们普遍认识到，模型在推理任务上的表现从根本上取决于所消耗的计算资源。因此，我们在不同的强化学习（RL）配置下训练了不同的专用模型，以促进开发出针对不同推理能力进行优化的模型。如表 2 所示，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 都支持三种特定的推理能力模式。对于每种模式，我们在强化学习训练过程中应用不同的长度惩罚和上下

文窗口，从而产生不同的推理输出标记长度。为了整合这些不同的推理模式，我们使用了由 `<think>` 和 `</think>` 标记界定的专用响应格式。此外，对于“Think Max”模式，我们在系统提示的开头添加了一个特定指令来引导模型的推理过程，如表 3 所示。

生成式奖励模型。通常，易于验证的任务可以使用简单的基于规则的验证器或测试用例进行有效优化。相比之下，难以验证的任务传统上依赖于人类反馈的强化学习（RLHF），这需要大量的人工标注来训练标量奖励模型。然而，在 DeepSeek-V4 系列的后训练阶段，我们摒弃了这些传统的基于标量的奖励模型。相反，为了解决难以验证的任务，我们精心策划了基于评价量规的强化学习数据，并采用生成式奖励模型（GRM）来评估策略轨迹。至关重要的是，我们将强化学习优化直接应用于生成式奖励模型本身。在这种范式中，行为者网络本身作为生成式奖励模型，能够同时优化模型的评估（判断）能力和其标准的生成能力。通过统一这些角色，模型的内部推理能力被固有地融合到其评估过程中，从而实现了高度稳健的评分。此外，由于模型利用自身的逻辑对复杂任务进行泛化，因此这种方法仅需极少量多样化的人工标注即可实现卓越的性能。

表 2 | 三种推理模式的比较

Reasoning Mode	Characteristics	Typical Use Cases	Response Format
Non-think	Fast, intuitive responses based on habits or simple rules.	Routine daily tasks, emergency reactions, low-risk decisions.	<code></think></code> summary
Think High	Conscious logical analysis, slower but more accurate.	Complex problem-solving, planning, medium-risk decisions.	<code><think></code> thinking tokens <code></think></code> summary
Think Max	Push reasoning to its fullest extent. Slow but powerful.	Exploring the boundary of model reasoning capability.	1. A special system prompt at the beginning. 2. <code><think></code> thinking tokens <code></think></code> summary

表 3 | 注入到系统提示中的 “Think Max” 模式指令。

Injected Instruction
Reasoning Effort: Absolute maximum with no shortcuts permitted. You MUST be very thorough in your thinking and comprehensively decompose the problem to resolve the root cause, rigorously stress-testing your logic against all potential paths, edge cases, and adversarial scenarios. Explicitly write out your entire deliberation process, documenting every intermediate step, considered alternative, and rejected hypothesis to ensure absolutely no assumption is left unchecked.

工具调用模式与特殊标记。与之前的版本一致,我们使用专用的 `<think></think>` 标签来描绘推理路径。在 DeepSeek-V4 系列中,我们引入了一种新的工具调用模式,该模式采用特殊的 “`|DSML|`” 标记,并使用基于 XML 的格式进行工具调用,如表 4 所示。我们的实验表明,XML 格式有效地减少了转义失败和工具调用错误,为模型与工具的交互提供了更为稳健的接口。

交错思维。DeepSeek-V3.2 引入了一种上下文管理策略,该策略保留了工具结果轮次中的推理痕迹,但在新用户消息到达时将其丢弃。虽然这种方法有效,但在复杂的代理工作流程中,它仍然造成了不必要的标记浪费——每次新用户轮次都会刷新所有累积的推理内容,迫使模型从头开始重建其问题解决状态。利用扩展的 1M 标记上下文,表 4 | DeepSeek-V4 系列的工具调用模式。

工具调用模式

##工具

您可以使用一组工具来帮助回答用户的问题。您可以通过编写如下“<|DSML|tool_calls>”块来调用工具：

```
<|DSML|tool_calls>
<|DSML|invoke name="$TOOL_NAME">
<|DSML|parameter name="$PARAMETER_NAME" string="true|false">$PARAMETER_VALUE
  </|DSML|parameter>
...
</|DSML|invoke>
<|DSML|invoke name="$TOOL_NAME2">
...
</|DSML|invoke>
</|DSML|tool_calls>
```

字符串参数应按原样指定，并设置‘string=“true”’。对于所有其他类型（数字、布尔值、数组、对象），请以JSON格式传递值，并设置‘string=“false”’。

如果启用了thinking_mode（由<think>触发），则您必须在调用任何工具或输出最终响应之前，在<think>...</think>内输出完整的推理过程。

否则，请在</think>后直接输出工具调用或最终响应。

###可用工具模式

[工具定义...]

您必须严格遵循上述定义的工具名称和参数模式来调用工具。

在 DeepSeek-V4 系列窗口中，我们进一步优化了这一机制，以最大限度地提高代理环境中交错思维的有效性：

- **工具调用场景。**如图 7(a) 所示，在整个对话过程中，所有推理内容都被完整保留。与 DeepSeek-V3.2 不同，后者在每个新用户回合时都会丢弃思维痕迹，而 DeepSeek-V4 系列则保留了所有轮次（包括跨越用户消息边界）的完整推理历史。这使得模型能够在长期代理任务中保持连贯、累积的思维链。
- **一般对话场景。**如图 7(b) 所示，保留了原始策略：当新用户消息到达时，丢弃之前轮次中的推理内容，以保持上下文的简洁性，适用于持续推

理痕迹提供有限益处的场景。

与 DeepSeek-V3.2 一样，通过用户消息模拟工具交互的代理框架（如 Terminus）可能不会触发工具调用上下文路径，因此可能无法从增强的推理持久性中受益。对于此类架构，我们仍建议使用非思维模型。

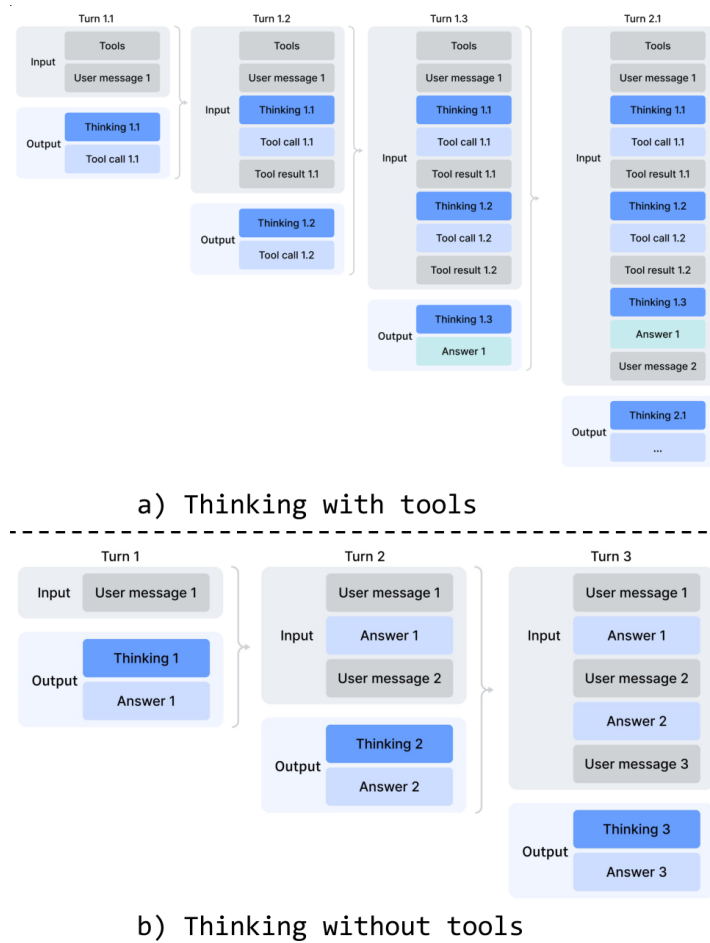


图 7 | DeepSeek-V4 系列的思维管理。

快速指令。在聊天机器人场景中，生成响应之前必须执行一系列辅助任务（如确定是否触发网络搜索、意图识别等）。传统上，这些任务由一个单独的小模型处理，由于无法重用现有的键值（KV）缓存，因此需要冗余的预填充。为了克服这一局限性，我们引入了快速指令。我们直接在输入序列后附加一组专用的特殊标记，其中每个标记对应一个特定的辅助任务。通过直接重用已计算的 KV 缓存，该机制完全避免了冗余的预填充，并允许某些任务（如生成搜索查询和确定权威性和领域）并行执行。因此，这种方法显著减

少了用户感知到的首次标记时间 (TTFT)，并消除了维护和迭代额外小模型的工程开销。表 5 总结了支持的快速指令标记。

5.1.2. 策略内蒸馏

在通过专门的微调和强化学习训练了多个领域特定的专家模型后，我们采用多教师策略内蒸馏 (OPD) 作为将专家能力融合到最终模型的主要技术。OPD 已成为一种有效的后训练范式，能够高效地将领域专家的知识 and 能力转移到单一、统一的模型中。这是通过让学生从教师模型在其自身生成的轨迹上的输出分布中学习来实现的。形式上，给定一组 N 个专家模型 $\{\pi_{E_1}, \pi_{E_2}, \dots, \pi_{E_N}\}$ ，OPD 目标函数定义为：

表 5 | 用于辅助任务的 Quick Instruction 特殊标记。

Special Token	Description	Format
<code>< action ></code>	Determines whether the user prompt requires a web search or can be answered directly.	<code>...< User >{prompt}< Assistant > <think>< action ></code>
<code>< title ></code>	Generates a concise conversation title after the first assistant response.	<code>...< Assistant >{response} < end_of_sentence >< title ></code>
<code>< query ></code>	Generates search queries for the user prompt.	<code>...< User >{prompt}< query ></code>
<code>< authority ></code>	Classifies the user prompt's demand for source authoritativeness.	<code>...< User >{prompt}< authority ></code>
<code>< domain ></code>	Identifies the domain of the user prompt.	<code>...< User >{prompt}< domain ></code>
<code>< extracted_url ></code> <code>< read_url ></code>	Determines whether each URL in the user prompt should be fetched and read.	<code>...< User >{prompt} < extracted_url >{url} < read_url ></code>

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^N w_i \cdot \text{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

在此公式中， W_I 表示为每位专家分配的权重，通常由专家的相对重要性决定。计算反向 KL 损失 $\text{D}_{\text{KL}}\theta\|_{EI}$ 需要从学生 π_{θ} 中采样训练轨迹，以保持策略内学习。其基本逻辑确保统一策略 θ 有选择地从与当前任务上下文相关的专

业专家（例如，对于数学推理任务与数学专家对齐，对于编程任务与编码专家对齐）中学习。通过此机制，来自物理上不同的专家权重的知识通过逻辑级别对齐被整合到统一的参数空间中，从而有效地避免了传统权重融合或混合强化学习技术中经常遇到的性能下降问题。在此阶段，我们使用了涵盖多个领域的十多个教师模型来蒸馏出一个单一的学生模型。

在处理上述开放预训练（Open Pre-training, OPD）目标时，先前的工作通常将全词汇 KL 损失简化为每个标记位置的标记级 KL 估计，并通过替换 $\text{sg} \log \frac{\pi_{E_t}(y_t | x_t, y_{<t})}{\pi_{\theta}(y_t | x_t, y_{<t})}$ （sg 表示停止梯度操作）作为策略损失计算中的每个标记的优势估计，来重用强化学习（Reinforcement Learning, RL）框架。尽管这种方法资源效率高，但它会导致梯度估计的方差大，并经常导致训练不稳定。因此，我们在 OPD 中采用了全词汇 logit 蒸馏。在计算反向 KL 损失时保留完整的 logit 分布，可以获得更稳定的梯度估计，并确保教师知识的忠实蒸馏。在接下来的小节中，我们将描述使全词汇 OPD 大规模可行的工程实践。

5.2. RL 和 OPD 基础设施

我们的后训练基础设施建立在为 DeepSeekV3.2 开发的可扩展框架之上。具体而言，我们集成了第 3.5 节中描述的相同分布式训练堆栈以及之前介绍的用于高效自回归采样的滚动引擎。在此基础上，我们在当前工作中引入了以下主要增强。这些设计使得涉及超过十个不同教师模型的超长上下文 RL 和 OPD 融合任务能够高效执行，从而显著加快了模型发布的迭代周期。

5.2.1. FP4 量化集成

我们应用 FP4（MXFP4）量化来加速部署和所有仅推理的前向传播，包括教师模型和参考模型的前向传播，从而减少内存流量和采样延迟。如第 3.4 节所述，我们在部署和推理阶段直接使用原生 FP4 权重。对于训练步骤，通过无损的 FP4 到 FP8 去量化步骤模拟 FP4 量化，从而无缝重用现有的 FP8 混合精度框架（主权重为 FP32），无需修改反向传播管道。

5.2.2. 全词汇按策略蒸馏 (OPD) 的高效教师调度

我们的框架支持全词汇按策略蒸馏 (OPD)，其中教师数量实际上不受限制，每个教师可能包含数万亿个参数。为实现这一目标，所有教师权重都被卸载到集中式分布式存储中，并在教师前向传播时按需加载，同时采用类似零往返 (ZeRO) 的参数分片来减轻 I/O 和 DRAM 压力。此外，在所有教师中，直接为词汇量 $|V| > 100k$ 的词汇表计算逻辑输出 (logits) 是禁止的，即使将其存储到磁盘也是如此。我们通过在向前传播期间仅将最后一层教师隐藏状态缓存到集中式缓冲区中来解决这一问题。在训练时，这些缓存状态会被检索并通过相应的预测头模块即时重建完整的逻辑输出。这种设计产生的重计算开销可以忽略不计，同时完全避免了与显式逻辑输出计算相关的内存负担。为了减轻教师预测头的 GPU 内存占用，我们在数据分发期间按教师索引对训练样本进行排序。这种安排确保每个不同的教师头在每个小批量中仅加载一次，并且在任何给定时间最多只有一个教师头驻留在设备内存中。所有参数和隐藏状态的加载/卸载操作都在后台异步进行，不会阻塞关键路径上的计算。最后，使用专门的 TileLang 内核计算教师和学生逻辑输出之间的精确 KL 散度，这加速了计算并减少了动态内存分配。

5.2.3. 可抢占且容错的 Rollout 服务

为了在最大化 GPU 资源利用率的同时，为高优先级任务提供快速的硬件配置，我们的 GPU 集群采用了集群范围内的抢占式任务调度器，任何正在运行的任务都可能在任何时候被抢占。此外，大规模 GPU 集群中硬件故障频发。为此，我们为 RL/OPD 的 Rollout 实现了一种可抢占且容错的 LLM 生成服务。

具体而言，我们为每个生成请求实现了一个令牌粒度的预写日志 (Write-Ahead Log, WAL)。每当为请求生成新令牌时，我们会立即将其附加到该请求的 WAL 中。在抢占期间，我们会暂停推理引擎并保存未完成请求的键值 (Key-Value, KV) 缓存。恢复时，我们使用持久化的 WAL 和保存的 KV 缓存来继续解码。即使在发生致命硬件错误时，我们也可以使用 WAL 中的持

久化令牌重新运行预填充阶段，以重建 KV 缓存。

重要的是，从零开始重新生成未完成的请求在数学上是错误的，因为这会引入长度偏差。由于较短的响应在中断后更有可能存活下来，因此从零开始重新生成会使模型在每次发生中断时更倾向于生成较短的序列。如果推理堆栈是批量不变的且具有确定性，那么通过为采样器中使用的伪随机数生成器设置一致的种子来重新生成，也可以解决这一正确性问题。然而，这种方法仍然会带来重新运行解码阶段的额外成本，使其效率远低于我们的基于标记的 WAL 方法。

5.2.4. 针对百万级标记上下文的扩展强化学习框架

我们针对百万级标记序列上的高效强化学习和在线预测部署（OPD）引入了有针对性的优化。在部署阶段，我们采用了一种可抢占且容错的部署服务，详见第 5.2.3 节。对于推理和训练阶段，我们将部署数据格式分解为轻量级元数据和每个标记的重量级字段。在数据分发过程中，可以加载整个部署数据的元数据以执行全局混洗和打包布局计算。每个标记的重量级字段通过共享内存数据加载器加载，以消除节点内数据冗余，并在小批量粒度上消耗后立即释放，从而显著降低 CPU 和 GPU 的内存压力。设备上的小批量数量根据工作负载动态确定，从而在计算吞吐量和 I/O 重叠之间实现有效的权衡。

5.2.5. 代理式 AI 的沙箱基础设施

为了满足代理式 AI 在后训练和评估期间多样化的执行需求，我们构建了一个生产级的沙箱平台——**DeepSeek 弹性计算（DSec）**。DSec 由三个 Rust 组件组成——API 网关（Apiserver）、主机代理（Edge）和集群监控器（Watcher）——它们通过自定义的 RPC 协议相互连接，并在 3FS 分布式文件系统（DeepSeek-AI, 2025）之上水平扩展。在生产环境中，单个 DSec 集群可管理数十万个并发沙箱实例。

DSec 的设计基于以下四个观察点：（1）代理工作负载高度异构，涵盖从

轻量级函数调用到具有不同操作系统和安全要求的完整软件工作流程；(2) 环境映像数量众多且体积庞大，但必须快速加载并支持迭代定制；(3) 高密度部署要求高效的 CPU 和内存利用率；(4) 沙箱生命周期必须与 GPU 训练计划相协调，包括抢占和基于检查点的恢复。基于这些观察点，我们将在下文分别详细阐述 DSec 的四个核心设计。

统一接口背后的四种执行基质。DSec 提供了一个单一的 Python 软件开发工具包 (SDK) (libdsec)，该工具包抽象了四种执行基质。**函数调用**将无状态调用分派到预热的容器池中，消除了冷启动开销。**容器**完全兼容 Docker，并利用 EROFS (Gao 等人, 2019) 按需加载功能实现高效的镜像组装。**microVM** 基于 Firecracker (Agache 等人, 2020) 构建，为安全敏感、高密度部署增加了虚拟机 (VM) 级别的隔离。**fullVM** 基于 QEMU (Bellard, 2005) 构建，支持任意客户操作系统。这四种基质共享一个通用的 API 接口——命令执行、通过分层存储实现快速镜像加载。DSec 通过分层、按需加载的方式，将快速启动与庞大且不断增长的环境镜像库相协调。对于容器，基础镜像和文件系统提交作为 3FS 支持的只读 EROFS 层直接挂载到 overlay lowerdirs 中。我们在挂载时将文件元数据保存在本地磁盘上；同时，数据块在请求时从 3FS 获取。对于 microVM，DSec 使用 overlaybd (Li 等人, 2020) 磁盘格式：只读基础层驻留在 3FS 上以实现跨实例共享，而写入则写入到本地写时复制层。此类快照是可链接的，有助于实现高效的版本控制和毫秒级恢复。

大规模并发环境下的密度优化。为了在每个集群中容纳数十万个沙箱，DSec 解决了两个资源瓶颈问题。首先，它缓解了虚拟化环境中的重复页面缓存占用，并应用内存回收技术以实现安全的超量使用。其次，它减轻了容器运行时中的自旋锁争用，从而降低了每个沙箱的 CPU 开销，显著提高了每个主机的打包密度。

轨迹记录与抢占安全恢复。DSec 为每个沙箱维护一个全局有序的轨迹日志，持久记录每个命令的调用及其结果。轨迹有三个用途：(1) **客户端快进**——

当训练任务被抢占时，沙箱资源仍得以保留；恢复时，DSec 重放缓存的先前已完成命令的结果，加速任务恢复，同时防止非幂等操作重新执行时出错；(2) **细粒度溯源**——每个状态变更的起源和相应结果均可追溯；(3) **确定性重放**——任何历史会话均可根据其轨迹忠实重现。

5.3. 标准基准评估

5.3.1. 评估设置

知识与推理。 知识与推理数据集包括 MMLU-Pro (Wang et al., 2024b)、GPQA (Rein et al., 2023)、Human Last Exam (Phan et al., 2025)、SimpleQA Verified (Haas et al., 2025)、Chinese-SimpleQA (He et al., 2024)、LiveCodeBench-v6 (Jain et al., 2024)、CodeForces (内部基准测试)、HMMT 2026 Feb、Apex (Balunovitch et al., 2025)、Apex Shortlist (Balunovitch et al., 2025)、IMOAnswerBench (Luong et al., 2025) 和 PutnamBench (Tsoukalas et al., 2024)。

对于代码任务，我们在 LiveCodeBench-v6 和一个内部 Codeforces 基准测试上评估了 DeepSeek-V4 系列。对于 Codeforces，我们收集了 14 场 Codeforces Division 1 比赛，共包含 114 道题目（2025 年 5 月至 2025 年 11 月）。Elo 评分计算方式如下。对于每场竞赛，我们为每个问题生成 32 个候选解决方案。对于每个问题，我们独立从中抽取 10 个解决方案，且不重复，并将它们随机排列以形成提交序列。每份提交都会根据领域专家构建的测试套件进行评判。已解决问题得分遵循 OpenAI (2025) 的罚分方案：模型获得与先前失败尝试次数相同的人类参赛者解决相同问题的中位数得分。这为每个抽取的提交序列生成了总竞赛得分，然后将其转换为竞赛排名，并随后通过标准的 Codeforces 评分系统转换为估计评分。竞赛级别的预期评分定义为对每个问题的 10 份提交的所有可能随机选择和排序的估计评分的期望值。模型的整体评分是所有 14 场竞赛中这些竞赛级别预期评分的平均值。

对于推理和知识任务，我们将温度设置为 1.0，对于非思考模式、高级

模式和最大模式，上下文窗口分别设置为 8K、128K 和 384K 个标记。对于数学任务（例如，HMMT、IMOAnswerBench、Apex 和 HLE），我们使用以下模板进行评估：“{问题} 逐步推理，并将你的最终答案放在 {} 内。”对于 DeepSeek-V4-Pro-Max 在数学任务上的表现，我们使用以下模板来引导更深层次的推理：“解决以下问题。问题可能要求你证明一个陈述，或者要求你给出答案。如果需要找到答案，你应该给出答案，并且你的最终解决方案也应该是对该答案有效性的严谨证明。{问题}”。

对于正式的数学任务，我们在 Lean v4.28.0-rc1 (Moura 和 Ullrich, 2021) 的代理环境中进行评估，该环境可访问 Lean 编译器和语义策略搜索引擎，在最大推理努力下运行多达 500 次工具调用。此外，我们还评估了一种计算量更大的流程，在该流程中，首先通过自验证 (Shao 等人, 2025) 生成并过滤候选的自然语言解决方案，然后将保留的解决方案作为指导提供给正式代理，以证明相应的 Lean 语句。这种设计使用非正式推理来改进探索，同时通过形式验证保持严格的正确性。只有在严格验证器 Comparator 在这两种设置下都接受提交结果时，才将其计为正确。

我们为 K2.6 和 GLM-5.1 留了一些空白条目，因为它们 API 太忙，无法对我们的查询做出响应。

1M-Token 上下文。由于 DeepSeek-V4 系列支持 1M-token 上下文，我们通过选择 OpenAI MRCR (OpenAI, 2024b) 和 CorpusQA (Lu 等人, 2026) 作为基准，在长上下文场景中评估模型性能。我们重新评估了 Claude Opus 4.6 和 Gemini 3.1 Pro 在这些任务上的表现，目的是标准化所有模型的配置。我们没有评估 GPT-5.4，因为其 API 未能对我们的大部分查询做出响应。

代理数据集包括 Terminal Bench 2.0 (Merrill 等人, 2026)、SWE-Verified (OpenAI, 2024e)、SWE Multilingual (Yang 等人, 2025)、SWE-Pro (Deng 等人, 2025)、BrowseComp (Wei 等人, 2025)、MCPAtlas 的公共评估集

(Bandi 等人, 2026)、GDPval-AA (AA, 2025; Patwardhan 等人, 2025) 以及 Tool-Decathlon (Li 等人, 2025)。

对于代码代理任务 (SWE-Verified、Terminal-Bench、SWE-Pro、SWE Multilingual)，我们使用内部开发的评估框架对 DeepSeek-V4 系列进行了评估。该框架提供了一套最基本工具——一个 bash 工具和一个文件编辑工具。最大交互步骤数设置为 500，最大上下文长度设置为 512K 个标记。关于 Terminal-Bench 2.0，我们承认 GLM-5.1 中指出的与环境相关的问题。尽管如此，为了保持一致性，我们报告了我们在原始 Terminal-Bench 2.0 数据集上的性能。在 Terminal-Bench 2.0 Verified 子集上，DeepSeek-V4-Pro 取得了约 72.0 的分数。

对于搜索代理任务 (BrowseComp、带工具的 HLE)，我们还使用了带有网络搜索和 Python 工具的内部工具，并将最大交互步骤设置为 500 步，最大上下文长度设置为 512K 个标记。对于 BrowseComp，我们采用了与 DeepSeek-V3.2 (DeepSeek-AI, 2025) 相同的丢弃所有上下文管理策略。

5.3.2. 评估结果

表 6 | DeepSeek-V4-Pro-Max 与闭源/开源模型的比较。“Max”、“xHigh”和“High”表示推理工作量。最佳结果以粗体突出显示；次佳结果以下划线标出。

Benchmark (Metric)		Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1-Pro High	K2.6 Thinking	GLM-5.1 Thinking	DS-V4-Pro Max
Knowledge & Reasoning	MMLU-Pro (EM)	<u>89.1</u>	87.5	91.0	87.1	86.0	87.5
	SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1	<u>57.9</u>
	Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	75.0	<u>84.4</u>
	GPQA Diamond (Pass@1)	91.3	<u>93.0</u>	94.3	90.5	86.2	90.1
	HLE (Pass@1)	<u>40.0</u>	39.8	44.4	36.4	34.7	37.7
	LiveCodeBench (Pass@1)	88.8	-	<u>91.7</u>	89.6	-	93.5
	Codeforces (Rating)	-	<u>3168</u>	<u>3052</u>	-	-	3206
	HMMT 2026 Feb (Pass@1)	<u>96.2</u>	97.7	94.7	92.7	89.4	95.2
	IMOAnswerBench (Pass@1)	<u>75.3</u>	91.4	81.0	86.0	83.8	<u>89.8</u>
	Apex (Pass@1)	34.5	<u>54.1</u>	60.9	24.0	11.5	38.3
	Apex Shortlist (Pass@1)	85.9	78.1	<u>89.1</u>	75.5	72.4	90.2
Long	MRCR 1M (MMR)	92.9	-	76.3	-	-	<u>83.5</u>
	CorpusQA 1M (ACC)	71.7	-	53.8	-	-	<u>62.0</u>
Agentic	Terminal Bench 2.0 (Acc)	65.4	75.1	<u>68.5</u>	66.7	63.5	67.9
	SWE Verified (Resolved)	80.8	-	<u>80.6</u>	80.2	-	<u>80.6</u>
	SWE Pro (Resolved)	57.3	57.7	<u>54.2</u>	58.6	<u>58.4</u>	55.4
	SWE Multilingual (Resolved)	77.5	-	-	<u>76.7</u>	73.3	<u>76.2</u>
	BrowseComp (Pass@1)	<u>83.7</u>	82.7	85.9	83.2	79.3	83.4
	HLE w/ tools (Pass@1)	<u>53.1</u>	52.0	51.6	54.0	50.4	48.2
	GDPval-AA (Elo)	<u>1619</u>	1674	1314	1482	1535	1554
	MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6	71.8	<u>73.6</u>
	Toolathlon (Pass@1)	47.2	54.6	48.8	50.0	40.7	<u>51.8</u>

表 6 展示了 DeepSeek-V4-Pro-Max 与其他闭源/开源模型的对比情况。此外，我们还评估了 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的不同模式，并将结果列于表 7 中。

知识。在通用世界知识的评估中，DeepSeek-V4-Pro-Max（DeepSeek-V4-Pro 的最大推理努力模式）在开源大型语言模型中树立了新的最高水平。如 SimpleQA-Verified 所示，DeepSeek-V4-Pro-Max 在所有现有开源基线模型中表现出色，绝对优势达到 20 个百分点。尽管取得了这些进步，它目前仍落后于领先的专有模型 Gemini-3.1-Pro。在教育知识和推理领域，DeepSeek-V4-Pro-Max 在 MMLU-Pro、GPQA 和 HLE 基准测试中略优于 Kimi 和 GLM，尽管它仍落后于领先的专有模型。总体而言，DeepSeek-V4-Pro-Max 是提升开源模型世界知识能力的重要里程碑。

此外，在基于知识的任务上，DeepSeek-V4-Flash 与 DeepSeekV4-Pro 之

间存在显著的性能差距；这是意料之中的，因为更大的参数数量有助于在预训练过程中更好地保留知识。值得注意的是，当分配更多的推理努力时，这两个模型在知识基准测试上的表现都有所提升。

表 7 | DeepSeek-V4 系列不同尺寸和模式之间的比较。“Non-Think”、“High” 和 “Max” 表示推理努力程度。

Benchmark (Metric)		DeepSeek-V4-Flash			DeepSeek-V4-Pro		
		Non-Think	High	Max	Non-Think	High	Max
Knowledge & Reasoning	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87.1	87.5
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2	57.9
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7	84.4
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37.7
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93.5
	Codeforces (Rating)	-	2816	3052	-	2919	3206
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95.2
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89.8
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38.3
	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5	90.2
Long	MRCR 1M(MMR)	37.5	76.9	78.7	44.7	83.3	83.5
	CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5	62.0
Agentic	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1	63.3	67.9
	SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4	80.6
	SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4	55.4
	SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1	76.2
	BrowseComp (Pass@1)	-	53.5	73.2	-	80.4	83.4
	HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7	48.2
	MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2	73.6
	GDPval-AA (Elo)	-	-	1395	-	-	1554
	Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51.8

推理能力。 DeepSeek-V4-Pro-Max 在推理基准测试中的表现优于所有先前的开源模型，并在多项指标上与最先进的闭源模型相当，而规模较小的 DeepSeekV4-Flash-Max 在代码和数学推理任务上也超越了之前的最佳开源模型 K2.6-Thinking。同时，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 在编程竞赛中表现出色。根据我们的评估，它们的性能与 GPT-5 相当，这是开源模型首次在这一任务上与闭源模型相匹敌。在 Codeforces 排行榜上，DeepSeek-V4-Pro-Max 目前在人类参赛者中排名第 23 位。DeepSeek-V4 在代理设置和计算密集型设置下的形式数学任务中也表现出色。在代理设置下，它取得了最优结果，如图 8 所示，超越了 Seed Prover (Chen 等人, 2025) 等先前的模型。在计算更为密集的流程下，性能进一步提升，超越了包括

Aristotle (Achim 等人, 2025) 在内的系统, 并在此设置下与已知最佳结果相当。

代理。 DeepSeek-V4 系列在评估中表现出强大的代理性能。对于代码代理任务, DeepSeek-V4-Pro 取得了与 K2.6 和 GLM-5.1 相当的结果, 尽管所有这些开源模型仍然落后于其闭源对应模型。在编码任务上, DeepSeek-V4-Flash 的表现不如 DeepSeek-V4-Pro, 尤其是在 Terminal Bench 2.0 上。在其他代理评估中也观察到了类似的趋势。值得注意的是, DeepSeek-V4-Pro 在 MCPAtlas 和 Toolathlon (两个包含广泛工具和 MCP 服务的评估测试集) 上的表现良好, 表明我们的模型具有出色的泛化能力, 并且不仅仅在内部框架上表现良好。



图 8 | 实际与前沿制度下的形式推理。左图: Putnam-200 Pass@8 按照 Seed-Prover 引入的设置, 评估 PutnamBench (Tsoukalas 等人, 2024) 的一个固定随机子集; 所有模型均在相同的问题集上进行测试。我们遵循 SeedProver 协议, 但用开源的 LeanExplore (Asher, 2025) 替换了专有搜索工具, 从而实现了代理工具最少且采样有界的轻量级设置。右图: Putnam-2025 在缩放的混合形式-非形式制度下探索数学推理的前沿, 其中非形式推理与形式验证相结合, 以暴露差距并提高严谨性; DeepSeek-V4 达到了完美证明的 120/120。

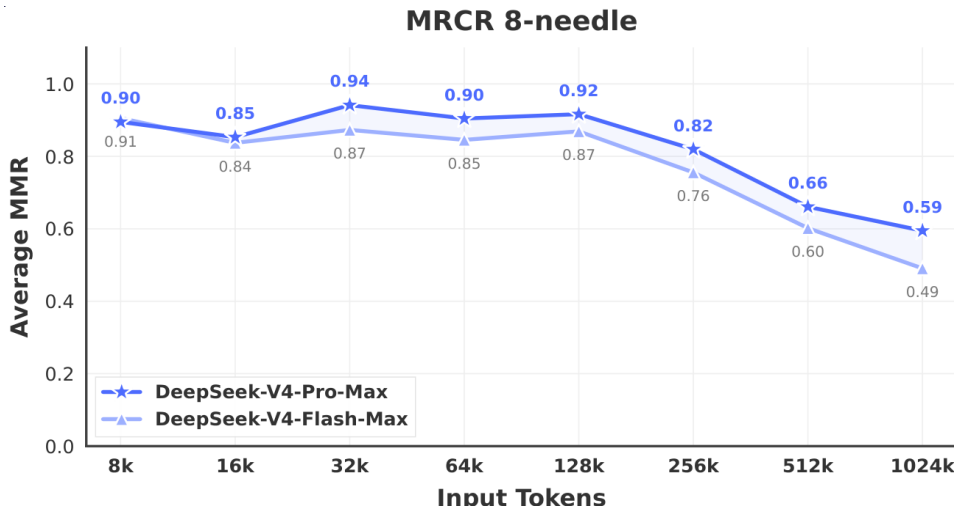


图 9 | DeepSeek-V4 系列在 MRCR 任务上的表现。

1M-Token 上下文。在衡量上下文检索能力的 MRCR 任务中，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro，但仍然落后于 Claude Opus 4.6。如图 9 所示，在 128K 上下文窗口内，检索性能保持高度稳定。虽然超过 128K 标记后性能开始下降，但与专有和开源模型相比，该模型在 1M 标记处的检索能力仍然非常强大。与 MRCR 不同，CorpusQA 与真实场景相似。评估结果还表明，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro。

推理努力。如表 7 所示，在强化学习中采用更长上下文和减少长度惩罚的 Max 模式在最具挑战性的任务上优于 High 模式。图 10 展示了 DeepSeek-V4-Pro、DeepSeekV4-Flash 和 DeepSeek-V3.2 在代表性推理和代理任务上的性能和成本比较。通过扩展测试时的计算能力，DeepSeek-V4 系列相较于前代产品取得了显著提升。此外，在像 HLE 这样的推理任务上，DeepSeek-V4-Pro 表现出比... 更高的标记效率

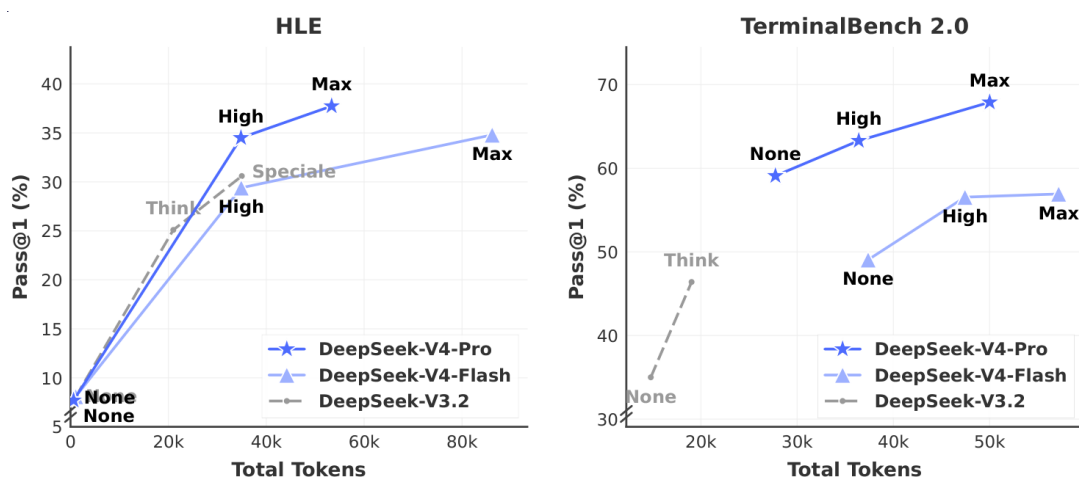


图 10 | 按推理工作量划分的 HLE 和 Terminal Bench 2.0 性能。“None”表示无思考模式，“Speciale”表示 DeepSeek-V3.2-Speciale 模型。

DeepSeek-V3.2.

5.4. 真实世界任务性能

标准化基准测试通常难以捕捉到复杂多样的真实世界任务，从而导致测试结果与实际用户体验之间存在差距。为了弥补这一差距，我们开发了专有的内

部指标，这些指标优先考虑真实世界的使用模式，而非传统的基准测试。这种方法确保了我们的优化能够转化为切实的收益。我们的评估框架专门针对 DeepSeek API 和 Chatbot 的主要用例，使模型性能与实际需求相一致。

5.4.1. 中文写作

DeepSeek 的主要用例之一是中文写作。我们对功能性写作和创造性写作进行了严格的评估。表 12 展示了 DeepSeek-V4-Pro 和 Gemini-3.1-Pro 在功能性写作任务上的成对比较。这些任务包括常见的日常写作查询，其中的提示通常简洁明了。我们选择 Gemini-3.1-Pro 作为基线，因为它是我们评估中表现最佳的中文写作外部模型。结果表明，DeepSeek-V4-Pro 的总体胜率达到 62.7%，优于基线的 34.1%；这主要是因为 Gemini 在中文写作场景中偶尔会允许其固有的风格偏好优先于用户的明确要求。

表 13 展示了创意写作的比较，该比较从两个维度进行评估：指令遵循和写作质量。与 Gemini-3.1-Pro 相比，DeepSeek-V4-Pro 在指令遵循方面的胜率达到 60.0%，在写作质量方面的胜率达到 77.5%，表明其在指令遵循方面略有提升，在写作质量方面有显著提高。尽管在总体用户案例分析中，DeepSeek-V4-Pro 取得了更优异的成绩，但当评估仅限于最具挑战性的提示时——特别是那些涉及高复杂性约束或多轮场景的提示——Claude Opus 4.5 的表现仍优于 DeepSeek-V4-Pro。如表 14 所示，Claude Opus 4.5 的胜率达到 52.0%，而 DeepSeek-V4-Pro 的胜率仅为 45.9%。

5.4.2. 搜索

搜索增强问答是 DeepSeek 聊天机器人的核心能力。在 DeepSeek 网站和应用程序中，“非思考”模式采用检索增强搜索（RAG），而“思考”模式则利用代理搜索。

检索增强搜索。我们对 DeepSeek-V4Pro 和 DeepSeek-V3.2 在客观和主观问答类别上进行了成对评估。如表 11 所示，DeepSeek-V4Pro 在两个类别上的

表现均大幅优于 DeepSeek-V3.2，显示出一致的优势。在单值搜索和规划与策略任务中观察到的提升最为显著，这表明 DeepSeek-V4Pro 在定位精确的事实答案和从检索到的上下文中合成结构化计划方面表现出色。然而，在比较和推荐任务上，DeepSeek-V3.2 仍然相对具有竞争力，这表明在需要对搜索结果进行平衡、多角度推理的场景中，DeepSeek-V4Pro 仍有改进的空间。

主动搜索。与标准检索增强生成（RAG）不同，主动搜索使模型能够针对每个查询迭代调用搜索和获取工具，从而显著提升整体搜索性能。在 DeepSeek-Chat 的思维模式中，我们优化了主动搜索功能，以在预定义的“思考预算”内最大限度地提高响应准确性。如表 9 所示，主动搜索在复杂任务上的表现始终优于 RAG。此外，其成本仍然非常高效，主动搜索的成本仅比标准 RAG 略高（见表 10）。

5.4.3. 白领任务

为了严格评估模型在复杂企业生产力场景中的实用性，我们构建了一套包含 30 个高级中文专业任务的全面组合。这些工作流程特意涵盖了高级认知需求，包括深入的信息分析、全面的文档生成和细致的文档编辑，横跨 13 个关键行业（如金融、教育、法律和技术）。评估是在配备基本工具（包括 Bash 和网页搜索）的内部代理工具中进行的。

鉴于这些任务的开放性，自动化指标通常难以捕捉到高质量回复的细微差别。因此，我们进行了人工评估，以比较 DeepSeek-V4-Pro-Max 与 Opus-4.6-Max 的性能。标注者从以下四个维度对模型输出进行了盲评：

- **任务完成：**核心问题是否已成功解决。
- **指令遵循：**是否遵守特定的约束和指令。
- **内容质量：**事实准确性、逻辑连贯性和专业语气。
- **格式美学：**布局可读性与视觉呈现。

如图 11 所示，DeepSeek-V4-Pro-Max 在多种中文白领任务上优于 Opus-4.6-Max，实现了高达 63% 的零损失率，并在分析、生成和编辑任务中展现出一致的优势。图 12 所示的详细维度分数凸显了该模型在任务完成和内容质量方面的主要优势。具体而言，DeepSeek-V4-Pro-Max 通过频繁提供补充见解和自我验证步骤，主动预测隐含的用户意图。此外，它在长文本生成方面表现出色，能够提供深入、连贯的叙述，而不是像 Opus-4.6-Max 那样频繁生成过于简化的要点。另外，该模型严格遵守正式的专业规范，如标准化的中文层次编号。然而，在指令遵循方面，它偶尔会忽视特定的格式约束，并在一定程度上落后于 Opus。此外，该模型在将大量文本输入精简为简洁摘要方面不够熟练。最后，在演示幻灯片的整体视觉设计方面，其格式美学仍有很大的改进空间。图 13、14 和 15 展示了几个测试案例；由于某些输出的长度较长，仅显示了部分页面。

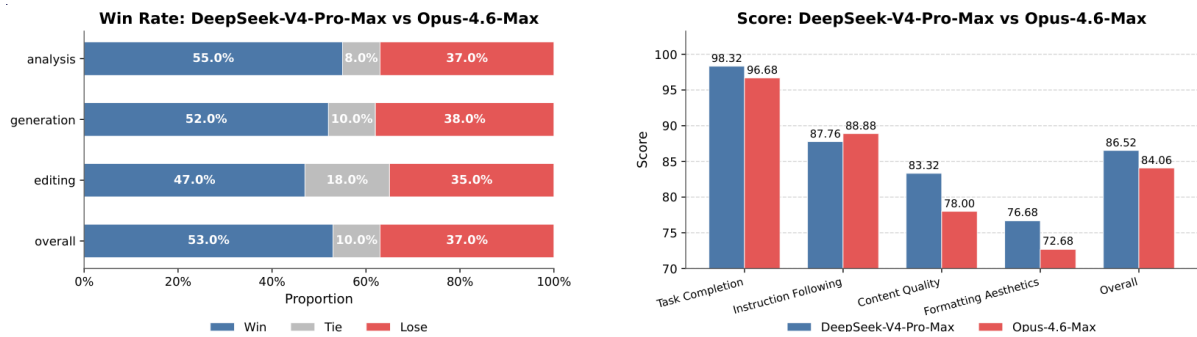


图 11 | 分析、生成、编辑任务以及整体表现中的胜率比较。

图 12 | 包括任务完成度、内容质量、格式美观度和遵循说明在内的详细维度评分。



图 13 | 为一个受欢迎的奶茶品牌和北京地铁起草联合营销提案的任务示例输出。

5.4.4. 代码代理

为了评估我们的代码代理能力，我们从真实的内部研发工作负载中精选了任务。我们从 50 多名内部工程师那里收集了 ~ 200 个具有挑战性的任务，涵盖了特征开发、错误修复、重构和诊断，涉及多种技术栈，包括 PyTorch、CUDA、Rust 和 C++。每个任务都附有其原始存储库、相应的执行环境和人工标注的评分标准；经过严格的质量筛选，保留了 30 个任务作为评估集。如表 8 所示，DeepSeek-V4-Pro 显著优于 Claude Sonnet 4.5，并接近 Claude Opus 4.5 的水平。

表 8 | 研发编码基准比较（严格包含外部模型以供评估）。

Model	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
Pass Rate (%)	13	47	67	70	73	80

在一项针对 DeepSeek 开发人员和研究人员（ $N=85$ ）的调查中——他们都曾在日常工作中使用过 DeepSeek-V4-Pro 进行代理编码——当被问及与

其他前沿模型相比，DeepSeek-V4-Pro 是否已准备好作为他们的默认和主要编码模型时，52% 的人表示同意，39% 的人倾向于同意，而不到 9% 的人表示不同意。受访者认为 DeepSeek-V4-Pro 在大多数任务中都能提供令人满意的结果，但也指出了其存在的微小错误、对模糊提示的误解以及偶尔的过度思考。

6. 结论、局限性与未来方向

在本研究中，我们展示了 DeepSeek-V4 系列的预览版，该系列针对下一代大型语言模型，旨在突破超长上下文处理的效率瓶颈。通过结合集成 CSA 和 HCA 的混合注意力架构，DeepSeek-V4 系列在长序列效率上实现了显著提升。架构创新与广泛的基础设施优化相结合，使得该系列能够高效地原生支持百万级标记上下文，并为未来的测试时扩展、长期任务以及在线学习等新兴范式奠定了必要的基础。评估结果表明，DeepSeek-V4-Pro-Max，即 DeepSeek-V4-Pro 的最大推理努力模式，重新定义了开放模型的最先进水平。它在知识基准测试上的表现显著优于之前的开源模型，实现了接近前沿专有模型的卓越推理性能，并展现了具有竞争力的代理能力。同时，DeepSeek-V4-Flash-Max 在保持高成本效益架构的同时，达到了与领先封闭模型相当的推理性能。我们相信，DeepSeek-V4 系列开创了开放模型百万级长度上下文的新时代，并为更高的效率、规模和智能铺平了道路。

为了追求极致的长上下文效率，DeepSeek-V4 系列采用了大胆的架构设计。为了降低风险，我们保留了许多经过初步验证的组件和技巧，这些组件和技巧虽然有效，但使得架构相对复杂。在未来的迭代中，我们将进行更全面、更有原则性的研究，以提炼出架构中最本质的设计，使其在不牺牲性能的情况下更加优雅。同时，尽管预期路由（Anticipatory Routing）和 SwiGLU 钳制（SwiGLU Clamping）已被证明在缓解训练不稳定性方面有效，但其基本原理仍不够明确。我们将积极研究训练稳定性的基础问题，并加强内部指标监控，旨在探索一种更具原则性和预测性的方法，以实现稳定的大规模训练。

此外，除了 MoE（混合专家）和稀疏注意力架构外，我们还将积极探索沿新维度的模型稀疏性，例如更稀疏的嵌入模块（Cheng 等人，2026），以在不牺牲能力的前提下进一步提高计算和内存效率。我们还将持续研究低延迟架构和系统技术，使长上下文部署和交互更具响应性。此外，我们认识到长时、多轮代理任务的重要性和实用价值，并将继续在这一方向上进行迭代和探索。我们还在努力将多模态能力融入我们的模型中。最后，我们致力于开发更好的数据整理和合成策略，以在日益广泛的场景和任务中不断提升模型的智能、鲁棒性和实用价值。

参考文献

AA. Gdpval-aa leaderboard, 2025. URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa>.

T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.

A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI’ 20, page 419-434, USA, 2020. USENIX Association. ISBN 9781939133137.

O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL <https://neurips.cc/virtual/2025/poster/119124>.

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL

<https://arxiv.org/abs/2506.11085>.

Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL <https://doi.org/10.48550/arXiv.2507.20534>.

Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639-3664, 2025b.

M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.

C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.

F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings

of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.

I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL <https://openreview.net/forum?id>

J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL <https://arxiv.org/abs/2512.17260>.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.

T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578-594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.

A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A com-

prehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601.07372. URL <https://doi.org/10.48550/arXiv.2601.07372>.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL <https://doi.org/10.48550/arXiv.2401.06066>.

T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL <https://pytorch.org/blog/flash-decoding/>.

L. De Moura and N. Bjørner. Z3: an efficient smt solver. In Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08, page 337-340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.

DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024. URL <https://doi.org/10.48550/arXiv.2406.11931>.

DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL <https://doi.org/10.48550/arXiv.2412.19437>.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL <https://doi.org/10.48550/434>.

DeepSeek-AI. Fire-flyer file system, 2025. URL <https://github.com/deepseek-ai/3FS>.

DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nat., 645(8081):633-638, 2025. URL <https://doi.org/10.1038/s41586-025-09422-z>.

DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.

X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.

H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. arXiv preprint arXiv:2404.10830, 2024.

X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba: A hybrid-head architecture for small language models. In The Thirteenth International Conference on Learning Representations, 2025. URL <https://openreview.net/forum?id=A1ztozypga>.

X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025.

D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368-2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL <https://doi.org/10.18653/v1/n19-1246>.

X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly readonly file system for resource-scarce devices. In Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19, page 149-162, USA, 2019. USENIX Association. ISBN 9781939133038.

A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? CoRR, abs/2406.04127, 2024. URL <https://doi.org/10.48550/arXiv.2406.04127>.

F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=pEWAcj>

L. Haas, G. Yona, G. D’Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.

Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.

D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint

arXiv:2009.03300, 2020.

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL <https://doi.org/10.48550/arXiv.2504.10356>.

B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.

K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.

M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601-1611, Vancouver, Canada, July 2017. Association for Computational

Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.

H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727-740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL <https://www.usenix.org/conference/atc20>. huiba.

H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.

Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=1NdN7eXyb4>.

J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL <https://doi.org/10.48550/arXiv.2502.16982>.

I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi: 10.64434/tml.20251026. <https://thinkingmachines.ai/blog/on-policy-distillation>.

policy-distillation.

Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. arXiv preprint arXiv:2601.14952, 2026.

T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.

M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.

MiniMax. Meet minimax-m2, 2025. URL <https://github.com/MiniMax-AI/MiniMax-M2>.

L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In International Conference on Automated Deduction, pages 625-635. Springer, 2021.

Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/K^2)$. Soviet Mathematics Doklady, 27:372-376, 1983.

NVIDIA Corporation. cublas documentation, 2024. URL <https://docs.nvidia.com/cuda/cublas/>. Version 12.4. Accessed: 2024-09-16.

OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL <https://huggingface.co/datasets/openai/MMMLU>.

OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL <https://huggingface.co/datasets/openai/mrcr>.

OpenAI. Learning to reason with llms, 2024c. URL <https://openai.com/index/learning-to-reason-with-llms>.

OpenAI. Introducing SimpleQA, 2024d. URL <https://openai.com/index/introducing-simpleqa/>.

OpenAI. Introducing SWE-bench verified we’ re releasing a human-validated subset of swebench that more, 2024e. URL <https://openai.com/index/introducing-swe-bench-verified/>.

OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL <https://doi.org/10.48550/arXiv.2508.10925>.

M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pages 429-431, 2023.

T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’ s last exam. arXiv preprint arXiv:2501.14249, 2025.

W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, volume EMNLP 2020 of Findings of ACL, pages 2401-2410. Association for Computational Linguistics, 2020. URL <https://doi.org/10.18653/v1/2020.findings-emnlp.217>.

Qwen. Qwen3 technical report. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL <https://doi.org/10.48550/arXiv.2505.09388>.

S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1-16. IEEE, 2020.

J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL <https://arxiv.org/abs/2112.08429>.

D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram' e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Statnczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson, B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. ChoquetteChoo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozi' nska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron,

G. Martins, H. Hashemi, H. Klimczak-Pluci' nska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui,

L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gorner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118, 2024.

S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 17555-17566, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076Abstract.html>.

B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour,

- L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.
- K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
- Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL <https://arxiv.org/abs/2511.22570>.
- N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL <http://arxiv.org/abs/1911.02150>.
- N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.
- F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL <https://openreview.net/forum?id=fR3wGCk-IXp>.
- J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.
- M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.
- G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL <https://arxiv.org/abs/2407.11214>.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *CoRR*, abs/2408.15664, 2024a. URL <https://doi.org/10.48550/arXiv.2408.15664>.

L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In *The Fourteenth International Conference on Learning Representations*, 2026.

Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *CoRR*, abs/2406.01574, 2024b. URL <https://doi.org/10.48550/arXiv.2406.01574>.

J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.

Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifoldconstrained hyper-connections, 2026. URL <https://arxiv.org/abs/2512.24880>.

L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762-4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL <https://doi.org/10.18653/v1/2020.coling-main.419>.

J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791-4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL <https://doi.org/10.18653/v1/p19-1472>.

C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, page 446-461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL <https://doi.org/10.1145/3731569.3764823>.

S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S.

Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL <https://arxiv.org/abs/>

C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>, 2025.

W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL <https://doi.org/10.48550/arXiv.2304.06364>.

D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL <https://openreview.net/forum?id=9FqARW7dwB>.

X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? arXiv preprint arXiv:2412.14689, 2024.

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL <https://openreview.net/forum?id=YrycTjllL0>.

附录

A. 作者列表和致谢

A.1. 作者列表

作者按姓氏字母顺序排列。标有 * 的姓名表示已离开我们团队的人员。

研究与工程: Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang*, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chenze Shao, Chong Ruan*, Conner Sun, Damai Dai, Daya Guo*, Dejian Yang, Deli Chen, Donghao Li, Erhang Li, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei*, Haotian Yuan, Haowei Zhang*, Haowen Luo, Haoyu Chen, Jie Wen Hu, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran*, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang*, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, M.S. Di, M.Y. Xu, Max Mei, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi

Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W. Zhang, Wangding Zeng, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xuheng Lin, Xuwei Fu, Y.C. Yan, Y.Q. Wang*, Y.W. Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Ying He, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuhan Wu, Yuhao Meng, Yuheng Zou, YuKun Li, Yunfan Xiong, Yupeng Chen, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, Z.F. Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang*, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao。

业务与合规部门：陈晨凌、侯成宇、季东杰、方伟、张恒清、罗佳、宋佳、蔡佳璐、梁健、周江亭、杨杰宇、陈进、周静子、郑俊敏、夏乐怡、朱琳燕、王妙军、李明明、韩敏敏、王宁、王盼盼、张鹏、陈如意、孙尚棉、吴少清、肖伟龙、安伟、侯文清、王先祖、孙晓文、王晓翔、张新宇、陈雪银、徐瑶、邵毅、马一玲、唐颖、杨越寒、徐悦、查玉坤、林玉平、闫玉婷、张泽凯、鞠哲、高哲人、吴中宇、曲子华、万子怡。

A.2. 致谢

我们衷心感谢 Dolly Deng 和其他测试人员对 DeepSeek-V4 系列模型性能提出的宝贵建议和反馈。

B. 评估详情

表 9 | DeepSeek-V4-Pro 的主动搜索与检索增强搜索对比。

Difficulty	Category	#	Agent Win	RAG Win	Tie	Agent%	RAG%	Tie%
Easy	Objective Q&A (客观问答)	196	110	43	43	56.1	21.9	21.9
	Subjective Q&A (主观问答)	321	198	56	67	61.7	17.4	20.9
Hard	Objective Q&A (客观问答)	168	102	33	33	60.7	19.6	19.6
	Subjective Q&A (主观问答)	184	126	27	31	68.5	14.7	16.8
Total (总计)		869	536	159	174	61.7	18.3	20.0

表 10 | 成本对比：DeepSeek-V4-Pro 的主动搜索与检索增强搜索（平均值）。主动搜索中的大多数工具调用是并行进行的。

Version	Tool Calls	Prefill (tokens)	Output (tokens)
V4 Agentic Search	16.2	13649	1526
V4 Retrieval Augmented Search	—	10453	1308

表 11 | DeepSeek-V4-Pro 与 DeepSeek-V3.2 在搜索问答任务上的对比评估。

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			V4 win	V3.2 win	tie	V4%	V3.2%	tie%
Objective Q&A (客观问答)	Single-value Search (单值信息查找)	95	36	10	49	37.9	10.5	51.6
	Entity Search (实体信息查找)	99	24	7	68	24.2	7.1	68.7
	Enumerative Search (枚举型信息查找)	95	19	8	68	20.0	8.4	71.6
	Subtotal (小计)	289	79	25	185	27.3	8.7	64.0
Subjective Q&A (主观问答)	Causal Analysis (原因分析)	100	28	5	67	28.0	5.0	67.0
	Comparison (对比)	96	28	20	48	29.2	20.8	50.0
	Advice Seeking (寻求建议)	92	23	8	61	25.0	8.7	66.3
	Recommendation (推荐)	95	26	19	50	27.4	20.0	52.6
	Planning & Strategy (攻略计划)	92	32	11	49	34.8	12.0	53.3
	Opinion & Evaluation (评价看法)	96	30	8	58	31.2	8.3	60.4
	Trend Analysis (趋势分析)	96	23	3	70	24.0	3.1	72.9
Subtotal (小计)		667	190	74	403	28.5	11.1	60.4
TOTAL (总计)		956	269	99	588	28.1	10.4	61.5

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			DS win	Gem win	Tie	DS%	Gem%	Tie%
Business Writing (办公文本)	Report (报告)	527	350	162	15	66.41	30.74	2.85
	Proposal (方案策划)	291	181	103	7	62.20	35.40	2.41
	Education (教育培训)	159	100	56	3	62.89	35.22	1.89
	Email & Letter (邮件书信)	146	107	37	2	73.29	25.34	1.37
	Notice (通知公告)	72	43	24	5	59.72	33.33	6.94
	Professional (专业文本)	63	34	27	2	53.97	42.86	3.17
	Recruitment (招聘求职)	42	27	15	0	64.29	35.71	0.00
	Technical (技术文本)	29	22	7	0	75.86	24.14	0.00
	Review (介绍评价)	20	15	5	0	75.00	25.00	0.00
Subtotal (小计)		1349	879	436	34	65.16	32.32	2.52
Media Writing (媒体文本)	Social Media (社交媒体文案)	267	156	101	10	58.43	37.83	3.75
	Ad Copy (广告商品文案)	214	109	98	7	50.93	45.79	3.27
	Long-form Content (内容平台长文)	99	71	25	3	71.72	25.25	3.03
	News Report (新闻报道)	51	27	22	2	52.94	43.14	3.92
	Advertorial (营销软文)	17	12	4	1	70.59	23.53	5.88
	Headline (标题)	11	7	4	0	63.64	36.36	0.00
	Narration Script (口播文案)	4	2	1	1	50.00	25.00	25.00
	Comment (评论)	3	2	1	0	66.67	33.33	0.00
Subtotal (小计)		666	386	256	24	57.96	38.44	3.60
Everyday Writing (生活文本)	Congratulatory (祝贺文本)	101	54	41	6	53.47	40.59	5.94
	Communication (沟通回复)	100	71	26	3	71.00	26.00	3.00
	Reflection (心得感想)	90	68	17	5	75.56	18.89	5.56
	Review (介绍评价)	55	44	9	2	80.00	16.36	3.64
	Comment (评论)	44	34	8	2	77.27	18.18	4.55
Subtotal (小计)		390	271	101	18	69.49	25.90	4.62
Oral Writing (口头文本)	Speech (发言稿)	226	135	85	6	59.73	37.61	2.65
	Narration Script (口播文案)	51	25	23	3	49.02	45.10	5.88
	Sales Script (话术)	31	22	6	3	70.97	19.35	9.68
	Dialogue (对话文本)	10	4	6	0	40.00	60.00	0.00
	Congratulatory (祝贺文本)	1	1	0	0	100.00	0.00	0.00
Subtotal (小计)		319	187	120	12	58.62	37.62	3.76
Official Document (公文文本)	Administrative Doc (事务文书)	117	60	53	4	51.28	45.30	3.42
	Personal Doc (个人文书)	73	45	27	1	61.64	36.99	1.37
	Government Doc (行政公文)	34	19	14	1	55.88	41.18	2.94
	Speech (发言稿)	3	1	2	0	33.33	66.67	0.00
	Essay Writing (申论写作)	3	1	1	1	33.33	33.33	33.33
Subtotal (小计)		230	126	97	7	54.78	42.17	3.04
Academic Writing (学术文本)	Research Paper (学术论文)	104	67	32	5	64.42	30.77	4.81
	Coursework (课程作业)	90	53	35	2	58.89	38.89	2.22
	Academic Support (学术辅助)	15	11	3	1	73.33	20.00	6.67
	Science Outreach (专业科普)	7	6	1	0	85.71	14.29	0.00
Subtotal (小计)		216	137	71	8	63.43	32.87	3.70
Total (总计)		3170	1986	1081	103	62.65	34.10	3.25

表 13 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文创意写作中的对比分析。

Subcategory (文体)	#	Instruction Following(指令遵循)						Writing Quality (写作质量)					
		DS	Gem	Tie	DS%	Gem%	Tie%	DS	Gem	Tie	DS%	Gem%	Tie%
Fiction (小说故事)	836	504	323	5	60.58	38.82	0.60	672	157	3	80.77	18.87	0.36
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.45	467	194	0	70.65	29.35	0.00
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.74	338	67	1	83.25	16.50	0.25
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.50	161	40	1	79.70	19.80	0.50
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.17	141	30	0	82.46	17.54	0.00
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.81	88	36	0	70.97	29.03	0.00
Prose (散文)	112	74	38	0	66.07	33.93	0.00	92	20	0	82.14	17.86	0.00
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.00	86	26	0	76.79	23.21	0.00
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.00	39	9	0	81.25	18.75	0.00
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.00	32	11	0	74.42	25.58	0.00
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.00	16	14	0	53.33	46.67	0.00
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.00	18	9	0	66.67	33.33	0.00
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.00	17	7	0	70.83	29.17	0.00
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.00	15	8	0	65.22	34.78	0.00
General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.00	5	4	0	55.56	44.44	0.00
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.00	4	2	0	66.67	33.33	0.00
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.00	5	0	0	100.00	0.00	0.00
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.00	2	0	0	100.00	0.00	0.00
Total (总计)	2837	1703	1119	15	60.03	39.44	0.53	2198	634	5	77.48	22.35	0.18

表 14 | DeepSeek-V4-Pro 与 Claude-Opus-4.5 在复杂指令遵循和多轮写作方面的对比。

Category	#	Internal Evaluation (内部综合评估)					
		DS	Opus	Tie	DS%	Opus%	Tie%
Complex Inst. Following (复杂指令跟随)	49	23	26	0	46.9%	53.1%	0.0%
Multi-Turn Writing (多轮写作)	147	67	76	4	45.6%	51.7%	2.7%
Total (总计)	196	90	102	4	45.9%	52.0%	2.0%